University of Staffordshire

School of Digital, Technology, Innovation and Business

Design and Implementation of Real-Time Signal Monitoring, Data Logging, and Filtering Systems

OLUWAKOREDE SOLOMON BAMIDELE

Student ID: 25023666

Module Leader: Dr Anas Amjad

ELEC73150 – Advanced Embedded Systems

May 2026

Table of Contents

List of Abbreviations

Abbreviation	Full Meaning
ABE	Attribute-Based Encryption
ADC	Analogue-to-Digital Converter
ADAS	Advanced Driver-Assistance Systems
AI	Artificial Intelligence
AV	Autonomous Vehicle
CAN	Controller Area Network
CAV	Connected and Autonomous Vehicle
DoS	Denial of Service
ECU	Electronic Control Unit
FIR	Finite Impulse Response
GNSS	Global Navigation Satellite System
GPIO	General-Purpose Input/Output
GRU	Gated Recurrent Unit
HAL	Hardware Abstraction Layer
HSM	Hardware Security Module
IIR	Infinite Impulse Response
IDS	Intrusion Detection System
IoT	Internet of Things
ISR	Interrupt Service Routine
IVN	In-Vehicle Network
LED	Light-Emitting Diode
LIN	Local Interconnect Network
LiDAR	Light Detection and Ranging
LSTM	Long Short-Term Memory
MAC	Message Authentication Code
ML	Machine Learning
OTA	Over-the-Air
OBD-II	On-Board Diagnostics II
PUF	Physically Unclonable Function
PKI	Public Key Infrastructure
RMS	Root Mean Square

Abbreviation	Full Meaning
STM32F4	ARM Cortex-M4 Microcontroller Family
TPM	Trusted Platform Module
V2X	Vehicle-to-Everything
Vpp	Peak-to-Peak Voltage

List of Figures

Abstract

Modern vehicles are complex cyber-physical systems that must deliver deterministic real-time performance within increasingly adversarial digital environments. This report investigates the dual challenge of dependable embedded operation and robust cybersecurity across contemporary automotive architectures. A critical literature review identifies the principal security threats affecting in-vehicle networks, connected vehicle interfaces, ADAS platforms, and lifecycle engineering processes, evaluating state-of-the-art mitigation strategies including lightweight cryptography, hardware-rooted trust anchors, secure OTA mechanisms, intrusion detection systems, and formal verification, while examining the tension between security efficacy, functional safety, and resource constraints. Building on this foundation, the report presents the design, implementation, and experimental validation of two real-time embedded systems on the STM32F4 Discovery platform. Task 1 delivers a dual-channel signal monitoring and data-logging system that classifies analogue waveforms and computes statistical metrics under strict timing and memory constraints. Task 2 implements computationally efficient IIR filters to separate a mixed-frequency signal, demonstrating trade-offs between filter order, signal fidelity, and energy consumption. Experimental results confirm that both systems achieve deterministic timing and reliable performance suitable for safety-critical embedded applications.

Keywords: Embedded systems, real-time processing, automotive cybersecurity, digital signal processing, STM32F4, IIR filtering, in-vehicle networks, ISO 26262, signal classification.

1. Introduction

Embedded systems constitute the computational core of virtually every safety-critical function in the modern automobile. From powertrain management and anti-lock braking to electronic power steering, infotainment, and advanced driver-assistance systems (ADAS), these platforms must satisfy hard real-time deadlines, ISO 26262 functional-safety requirements, and severe constraints on memory, processing power, and energy availability [1]. A contemporary vehicle may contain over 100 Electronic Control Units (ECUs) executing tens of millions of lines of code across a heterogeneous mix of in-vehicle networks including Controller Area Network (CAN), Local Interconnect Network (LIN), FlexRay, and Automotive Ethernet, all interconnected through centralised gateways or emerging zonal compute units [3], [4], [7].

This expanded connectivity transforms the automobile into a highly connected cyber-physical system and substantially enlarges the attack surface. Historically isolated, proprietary embedded systems are now exposed to adversaries with sophisticated toolsets and global reach [3], [5], [8]. High-profile demonstrations including the 2015 Jeep Cherokee remote compromise, the TBONE Tesla Wi-Fi exploit, and GPS spoofing attacks on ADAS perception systems have established that automotive cyber-attacks pose direct safety risks, not merely reputational consequences [3], [5]. Regulatory frameworks including UNECE WP.29 Regulation 155 and ISO/SAE 21434:2021 now mandate cybersecurity engineering across the entire vehicle lifecycle, requiring co-engineering of safety and security [6], [7], [8].

Within this context, foundational embedded capabilities remain paramount. Real-time signal acquisition, accurate statistical analysis, and frequency-selective filtering are essential prerequisites for ensuring sensor data is captured faithfully and processed deterministically. Corrupted or spoofed signals propagating through control loops can produce unsafe vehicle behaviour. This report addresses these themes through two strands. First, a technical literature review identifies and critically analyses the major security challenges confronting automotive embedded systems and evaluates the most prominent mitigation techniques with explicit attention to real-time and resource constraints. Second, the design, implementation, and validation of two real-time embedded signal-processing systems on the STM32F4 Discovery platform are presented: Task 1 delivers a dual-channel signal monitoring and data-logging system, while Task 2 implements computationally efficient filters to separate co-channel mixed-frequency signals.

2. Literature Review

As vehicles evolve from mechanically dominated machines into software-defined cyber-physical systems, the security of their embedded platforms has become a first-order engineering concern. This chapter critically reviews the principal security challenges and evaluates state-of-the-art mitigation techniques, attending throughout to real-time constraints, functional-safety requirements, and limited computational resources.

2.1 Security Challenges in Automotive Embedded Systems

2.1.1 In-Vehicle Networks and Electronic Control Units

In-vehicle networks interconnect the ECUs that manage sensing, actuation, diagnostics, and coordination across every vehicle domain. Classical CAN, which remains the dominant bus for body, chassis, and powertrain ECUs, is a broadcast, identifier-based, unauthenticated protocol. Any node with bus access can transmit frames without source authentication, message confidentiality, or protection against replay and flooding. A compromised ECU obtained through physical OBD-II access or by lateral movement from a telematics unit can inject arbitrary CAN frames, mount spoofing attacks, or launch denial-of-service flooding that causes safety-critical ECUs to miss scheduling deadlines [4], [7]. Gateway architectures introduce a further concern: because a central gateway aggregates traffic from powertrain, chassis, infotainment, and telematics domains, a single compromised interface serves as a pivot point into safety-critical areas [3], [4].

Rathore et al. classify in-vehicle attack vectors into sensor-initiated, infotainment-initiated, telematics-initiated, and direct-interface-initiated categories, demonstrating that both wireless and physical pathways ultimately converge on the CAN backbone [4]. ECU firmware presents a further critical surface: many ECUs execute long-lived, monolithic firmware without code-signing or secure-boot enforcement. Insecure diagnostic interfaces and absent cryptographic integrity verification on software updates enable persistent firmware replacement and malware installation [3], [5]. Krichen emphasises that insecure communication channels, inadequate authentication, and structural protocol weaknesses constitute the foundational challenges that all higher-level security measures must address [7]. These weaknesses persist because IVN protocols were designed to optimise determinism and minimal overhead, properties that directly conflict with strong cryptographic protection, making retrofitting security into legacy protocols one of the most difficult challenges in automotive cybersecurity.

2.1.2 Connected Vehicles: V2X, IoT, and Cloud

Modern vehicles exchange data with roadside infrastructure, cloud services, mobile devices, and other vehicles through V2V, V2I, V2P, and V2N links delivered over DSRC/IEEE 802.11p, C-V2X, Wi-Fi, Bluetooth, and cloud APIs [2], [5], [8]. Sadhu et al. characterise IoT and IoV deployments as facing multi-layer threats across perception, network, processing, and application layers, with DDoS, Sybil attacks, man-in-the-middle interception, and VANET routing attacks particularly relevant to vehicular systems [2]. Wang et al. demonstrate that coordinated cyber-attacks on CAV connectivity can degrade not only

individual vehicle safety but also cooperative functions such as platooning and intersection management, with cascading consequences at the traffic-system level [5]. Privacy is a distinct but related concern: continuous generation of location telemetry, driving behaviour data, and biometric signals, combined with persistent identifiers in V2X certificate schemes, creates substantial risks of long-term tracking and surveillance that attract GDPR compliance obligations [2], [5], [8]. Critically, many connected-vehicle vulnerabilities arise from architectural assumptions inherited from an era when connectivity was not anticipated, with features added incrementally to legacy platforms never designed for continuous external communication.

2.1.3 Threats to ADAS and Autonomous Driving Platforms

Advanced perception in ADAS and autonomous vehicles relies on cameras, LiDAR, radar, GNSS and IMU sensors fused by machine-learning-based algorithms that feed planning and control systems operating at safety-critical timing margins [1], [5], [6], [8]. Sensor spoofing attacks target this perception layer directly: LiDAR spoofing can inject phantom obstacles, GPS spoofing can induce false position estimates, and camera blinding can saturate image sensors. Mehta et al. survey adversarial machine-learning threats, classifying attacks including patch-based perturbations of traffic signs and model-level attacks on object detection and segmentation models [1]. These attacks exploit deep neural network statistics to produce targeted misclassifications using imperceptible perturbations. Wang et al. demonstrate that such attacks propagate through the full CAV stack to degrade cooperative safety functions at the traffic-system level [5]. The safety-security interaction is acute: security mechanisms must not violate timing budgets or impair safety functions, yet a successful perception attack can directly cause catastrophic control outcomes. Sonko et al. identify cybersecurity as a primary barrier to large-scale AV deployment, noting that the interdependency between safety and security requirements makes independent optimisation of either infeasible [6].

2.1.4 Systemic and Lifecycle Challenges

Automotive ECUs typically operate in the 32 to 200 MHz range with kilobytes to low megabytes of RAM, executing RTOS kernels with tasks scheduled at sub-millisecond granularity. The computational overhead of cryptographic operations, the memory footprint of intrusion detection models, and the latency of security handshakes can all jeopardise schedulability [3], [4], [7]. Beyond resource constraints, the heterogeneous mix of ECUs from dozens of suppliers across multiple hardware generations makes uniform security deployment and retrofit both technically complex and economically challenging, particularly given vehicle lifespans of ten to fifteen years over which the threat landscape evolves while hardware remains fixed. Krichen emphasises that integrating formal verification across the development lifecycle is both necessary and practically challenging, particularly when real-time constraints must be encoded within the formal model [7].

2.2 Prominent Security Techniques

2.2.1 Lightweight Cryptography and Secure Communication

The most direct mitigation for CAN's lack of authentication is the addition of Message Authentication Codes to frame payloads. Given CAN's 8-byte payload constraint, automotive designs use truncated MACs of 24 to 32 bits generated using AES-128 in CMAC mode, standardised within the AUTOSAR SecOC specification [4], [7]. CAN-FD and Automotive Ethernet ease cryptographic integration by providing larger payloads and higher bandwidth, enabling full authenticated encryption without the overhead penalties on Classical CAN. For V2X communications, PKI schemes including ETSI ITS and IEEE 1609.2 provide authentication and pseudonymity through ECDSA signatures and rotating pseudonymous certificates [2], [5], [8]. Sadhu et al. compare PKI, ABE, MAC-based, ECC-based, and PUF-based schemes, demonstrating that no single approach dominates across security strength, computational cost, and key management complexity, motivating hybrid architectural designs [2].

2.2.2 Hardware-Rooted Security

Hardware Security Modules integrated within automotive SoCs provide isolated environments for cryptographic operations, AES and SHA acceleration, true random number generation, and secure boot chain enforcement. By executing MAC generation within the HSM rather than on the main application core, timing impact on safety-critical RTOS tasks is substantially reduced [3], [7]. Trusted Platform Modules extend this to include attestation, enabling cloud-based monitoring to detect compromised software states in gateways and zonal compute units [3]. Physical Unclonable Functions exploit silicon manufacturing variations to create device-unique identities without explicit key storage, substantially reducing the attack surface for physical key-extraction attacks. Kukkala et al. and Sadhu et al. identify PUF-based key derivation as particularly promising for resource-constrained automotive ECUs [2], [3].

2.2.3 Intrusion Detection Systems

Since cryptographic defences can be bypassed through compromised ECUs that possess legitimate session keys, intrusion detection systems provide an essential complementary layer. Kukkala et al. compare AI-driven IDS architectures for CAN traffic: GAN-based detectors encode frame sequences as image tensors and achieve high accuracy against both known and novel attack patterns; GRU-based recurrent autoencoders detect deviations by reconstruction error without requiring labelled attack data; LSTM encoder-decoder architectures combined with one-class SVMs demonstrate strong performance on injection and replay attacks; Temporal CNN with attention classifiers provide a computationally efficient alternative deployable on constrained ECUs [3]. For external vehicular networks, CNN-based detectors classify attacks on roadside units while LSTM autoencoders detect anomalies in V2X streams [3]. Rathore et al. note that ML-based IDSs can detect previously unseen attacks but must be carefully validated to control false-positive rates, since incorrectly suppressing legitimate safety-critical CAN messages can cause more immediate harm than the attack itself [4].

2.2.4 Multi-Layer Architectures, Formal Methods, and AI Robustness

The consensus in the literature is that robust protection requires defence-in-depth across multiple architectural layers. Rathore et al. propose a protocol-independent multi-layer framework combining ML-based IDS, cryptographic MAC, and port-centric access control across all ECU, gateway, and external communication interfaces [4]. Kukkala et al. advocate zero-trust assumptions for all inter-component communication and argue that emerging zonal and HPC architectures enable hardware-enforced domain isolation and address-space randomisation that limits lateral movement after initial compromise [3]. Formal methods including model checking with SPIN or UPPAAL and static analysis tools are increasingly applied to verify protocol properties and detect firmware vulnerabilities, though Krichen notes that tool scalability and encoding real-time constraints in formal models remain the primary open challenges [7]. For ADAS and AV perception, adversarial training with PGD or Carlini-Wagner perturbations and multi-modal sensor fusion substantially improve resilience against spoofing and adversarial examples [1], [5], [6]. Sonko et al. and Sadaf et al. emphasise that AI plays a dual role in AV security, constituting both the primary attack surface for adversarial ML and the most powerful available tool for anomaly detection and adaptive defence [6], [8].

2.3 Critical Evaluation

Several cross-cutting trade-offs constrain practical security design. Security versus real-time performance is the most fundamental: every security mechanism consumes processor cycles, memory bandwidth, and bus capacity simultaneously required by safety-critical control tasks. HSMs and TPMs substantially mitigate this tension but do not eliminate it, and formal schedulability analysis under adversarial conditions remains underexplored [3], [4], [7]. The safety versus security availability trade-off requires that IDS responses preserve core vehicle safety functions even when uncertain, demanding joint optimisation informed by FMEA and fault-tree analysis under ISO 26262 [1], [3], [4]. Multi-layer AI-driven defences also increase compositional complexity, making formal assurance harder to achieve [3], [7]. Finally, legacy and incremental deployment constraints mean that the most effective techniques are available only in newly designed vehicles, while the existing fleet will continue operating for a decade or more, requiring retrofitted gateway-based defences as imperfect intermediate strategies [3], [4], [8]. The most mature direction is compositional defence-in-depth security, integrating lightweight cryptography, hardware roots of trust, AI-based IDS, secure OTA, and formal methods, with the integration of HARA under ISO 26262 with TARA under ISO/SAE 21434 representing the critical methodological frontier.

3. System Design, Implementation and Analysis

3.1 Task 1: Real-Time Signal Monitoring and Data Logging

3.1.1 System Architecture and Design Rationale

Task 1 required the STM32F4 Discovery Kit to acquire, statistically analyse, classify, and log analogue signals from two independent channels. The system had to process two independent analogue channels, detect whether each channel carried a 1 kHz sinusoidal signal, a 1 kHz square wave, or no signal, and compute statistical metrics every second. Additionally, the system needed to computing peak-to-peak voltage (Vpp), mean, variance, standard deviation, and RMS every second, logging the first ten signal cycles and fifteen seconds of summary statistics, and providing LED-based system state indication. The architecture was structured around three tightly integrated subsystems: a real-time analogue acquisition subsystem driven by timer interrupts; a signal processing and statistical computation subsystem executed on complete buffer capture; and a system state indication and data-logging subsystem managing memory-bounded output. Figure 1 shows the STM32CubeMX pinout configuration used to realise this architecture, confirming ADC1, ADC2, TIM6, and GPIO port D assignment.

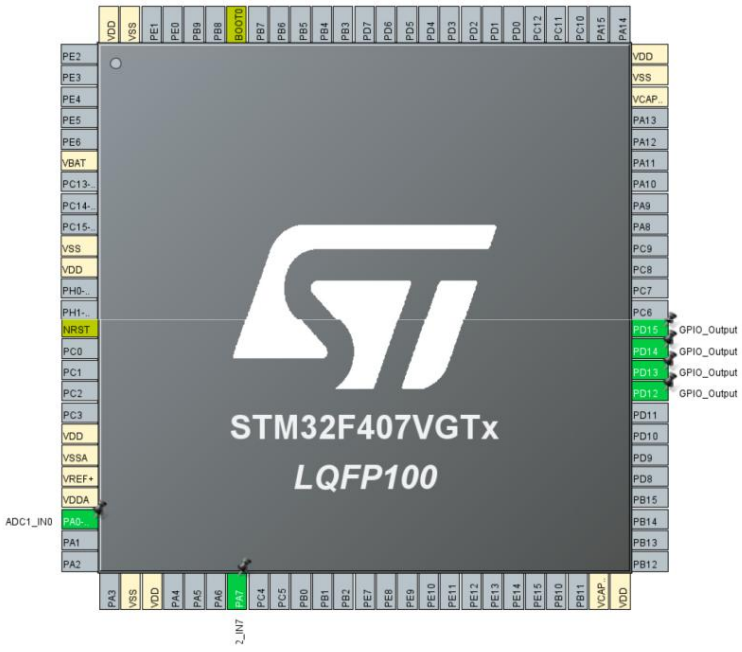


Figure 1 showing pinout view of the STM32CubeMX

A sampling frequency of 8 kHz was selected to satisfy the Nyquist criterion with adequate margin above the 1 kHz input signals, providing 8,000 samples per channel per second and thereby meeting the one-second update interval required. Static array allocation was employed throughout to eliminate heap fragmentation and guarantee deterministic memory access at compile time, consistent with MISRA-C automotive coding guidelines and the memory determinism requirements of ISO 26262.

3.1.2 Peripheral Configuration

As shown in the STM32CubeMX system view in Figure 2, ADC1 and ADC2 were configured for dual-channel sampling on pins PA0 and PA7 respectively, with TIM6 generating the 8 kHz periodic sampling trigger and GPIO Port D driving the LED indicators. Figure 3 presents the clock configuration, which is set to 168 MHz operation via the onboard HSE oscillator, which provides sufficient processing headroom for the statistical computation pipeline within each one-second acquisition window. All acquisition buffers and logging structures were implemented as fixed-size static arrays as shown below, ensuring deterministic memory access and preventing runtime allocation failures that could corrupt safety-relevant system state.

```
uint16_t ADC_DATA1[8000];  uint16_t ADC_DATA2[8000];
float    Volt1[8000];      float    Volt2[8000];
float    Vpp_log1[15];     float    Mean_log1[15];    float RMS_log1[15];
float    Vpp_log2[15];     float    Mean_log2[15];    float RMS_log2[15];
```

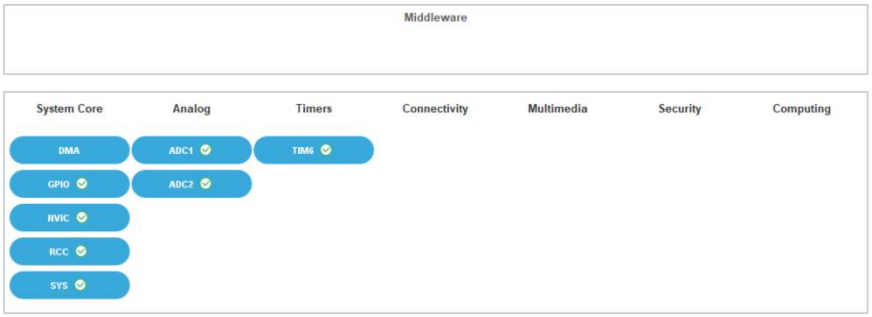


Figure 2 showing System view of the STM32CubeMX

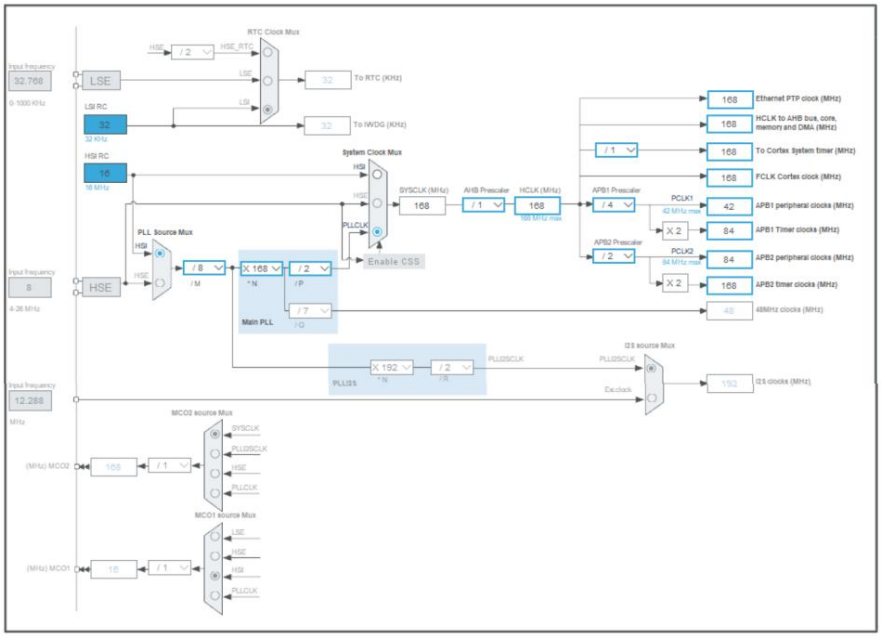


Figure 3 showing the clock configuration of the STM32CubeMX

3.1.3 Real-Time Acquisition and Signal Processing

3.1.3.1 Interrupt-Driven Sampling

The acquisition core is implemented inside the HAL_TIM_PeriodElapsedCallback interrupt service routine, which executes at 8 kHz under TIM6 control. The ISR starts both ADCs, polls for conversion completion, converts the 12-bit ADC values to voltages, and manages buffer indexing. The voltage conversion formula maps the full ADC range to the 0 to 3 V input domain as shown in the code below. At buffer index i = 4,000, corresponding to 500 ms, the blue LED toggles to provide a system heartbeat as seen in Figure 7. When i reaches 8,000, the buffer is full, all statistical processing functions are invoked sequentially, the logging arrays are updated, and i resets to zero. This single-threaded ISR model eliminates race conditions and maintains deterministic execution within the one-second cycle.

```
HAL_ADC_Start(&hadc1);  HAL_ADC_PollForConversion(&hadc1, 1);
HAL_ADC_Start(&hadc2);  HAL_ADC_PollForConversion(&hadc2, 1);
Volt1[i] = ADC_DATA1[i] * (3.0f / 4096.0f);
Volt2[i] = ADC_DATA2[i] * (3.0f / 4096.0f);
```

3.1.3.2 Statistical Metrics

Once the buffer is full, five metrics are computed for each channel independently. Vpp is computed by scanning all 8,000 samples for global maximum and minimum values and serves as the primary amplitude indicator and signal presence threshold. The mean quantifies waveform symmetry and DC offset, and is the primary discriminator for the no-signal condition. Variance and standard deviation quantify waveform spread: square waves exhibit higher variance than sinusoidal waves of the same amplitude due to their bimodal amplitude distribution, providing a secondary classification feature. RMS voltage provides the effective signal magnitude. The Vpp, mean and RMS plot of a validated sinusoidal input is shown in Figure 4.

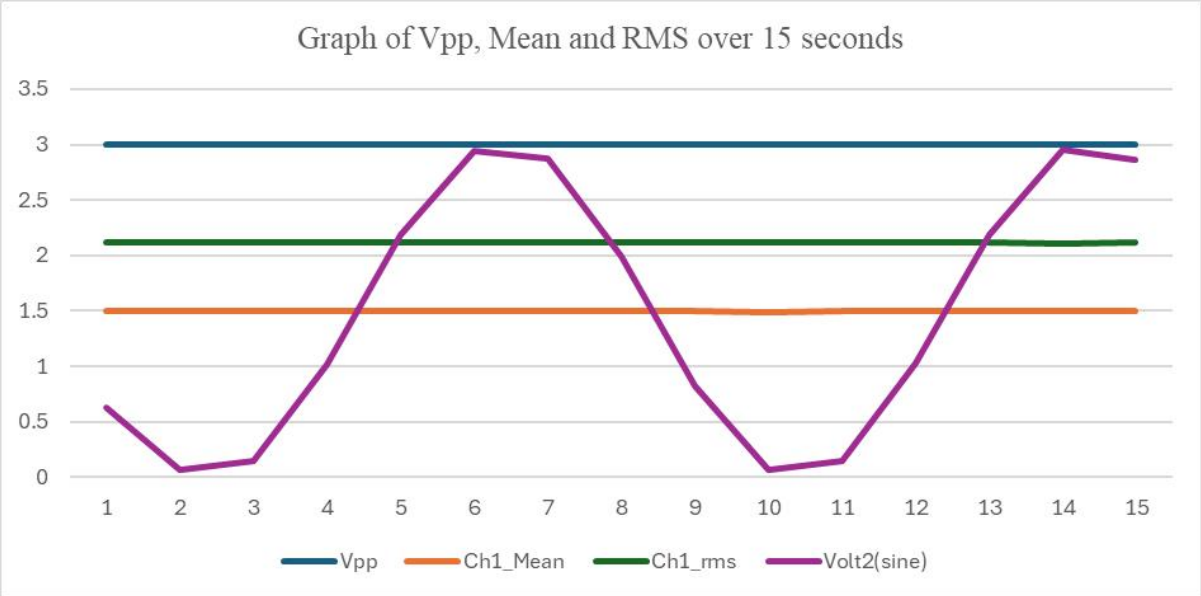


Figure 4 showing the Vpp, mean and RMS plot of a sinusoidal input over 15seconds

3.1.3.3 Signal Type Classification

The classification algorithm categorises each channel as sinusoidal (type = 22), square wave (type = 11), or no signal (type = 10) using a lightweight sequential decision process operating on the first 50 voltage buffer samples together with the computed mean and Vpp. The no-signal condition is detected first using a mean voltage threshold below 0.9 V. Square waves are identified by the combination of an abrupt inter-sample voltage transition exceeding 2.9 V and a Vpp greater than 2.5 V, exploiting the characteristic fast edges of square waves under 8 kHz sampling. Signals satisfying neither criterion are classified as sinusoidal. The classifier requires only scalar comparisons with no division or transcendental operations, making it computationally efficient for execution on the ARM Cortex-M4. The complete classification function is provided in Appendix C.

```
void Sig_type(void)
{
    // CHANNEL 1
    Float prev1 = Volt1[0];
    for (int i = 1; i < 50; i++)
    {
      float v1 = Volt1[i];
      float diff1 = fabsf(v1 - prev1);
      if (ch1_mean < 0.9f)             { type1 = 10; break; }  // No Signal
      if (diff1 > 2.9f && ch1_vpp > 2.5f){ type1 = 11; break; }  // Square Wave
      type1 = 22;                                              // Sinusoidal
      prev1 = v1;
}
```

3.1.4 LED Status Indication

Three onboard LEDs provide real-time hardware-level system state feedback without requiring a serial terminal. The blue LED on PD15 toggles every 500 ms as a system heartbeat. The green LED on PD12 illuminates when both channels carry valid signals (Vpp greater than 2.5 V on both channels), as shown in Figure 5 where both function generator outputs are active. The red LED on PD13 illuminates when one or both channels lack a valid signal, as shown in Figure 6. Table 1 summarises the complete LED logic.

Table 1: LED Status Indicators and Associated System Conditions

LED (Pin)	Condition	System Meaning
Blue (PD15)	Toggles every 0.5 s	System heartbeat: acquisition loop running
Green (PD12)	ch1_vpp and ch2_vpp both > 2.5 V	Both channels carry a valid signal
Red (PD13)	ch1_vpp or ch2_vpp < 2.5 V	One or both channels missing or no signal

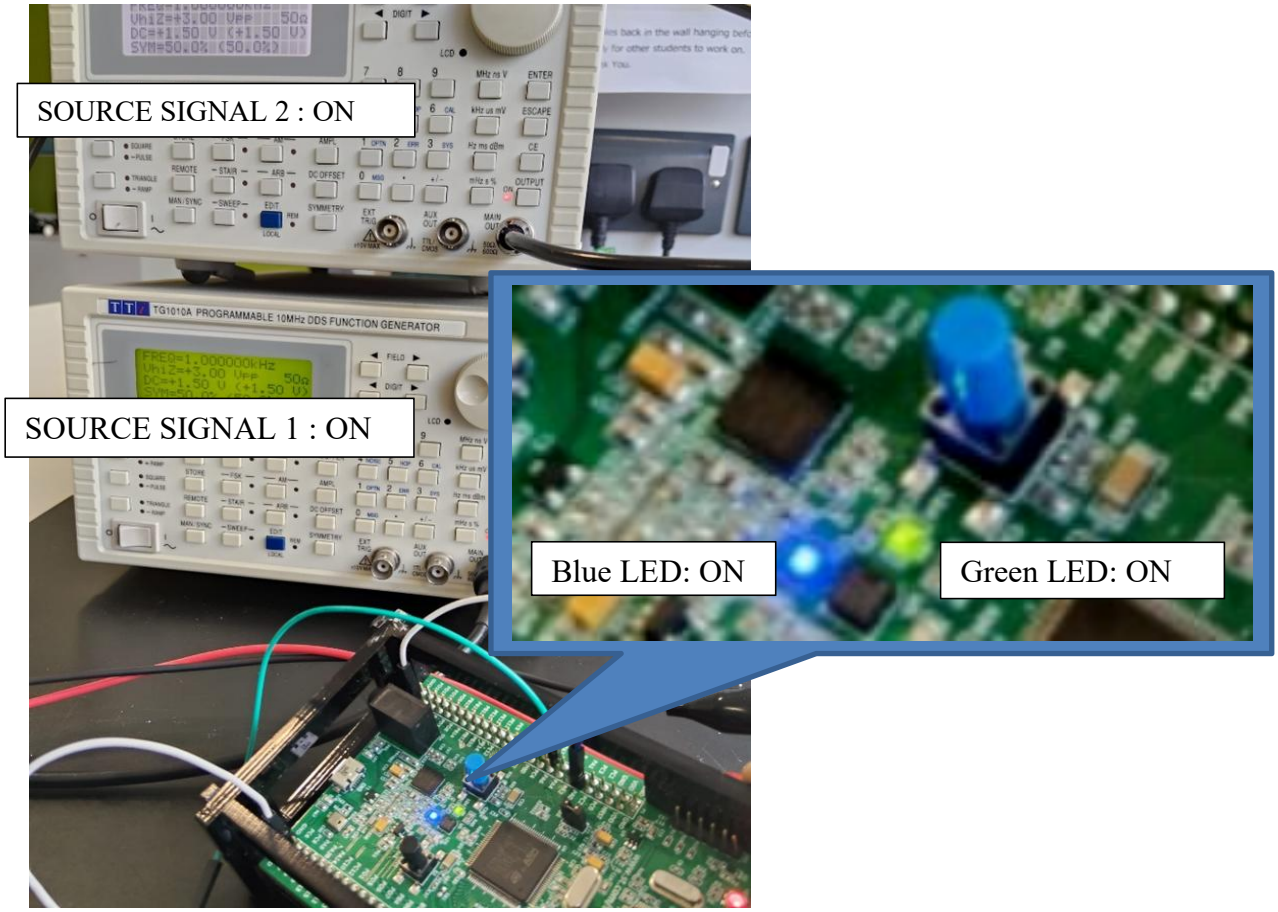


Figure 5 showing the blue and green LED ON indicating half a second blink and presence of both signals

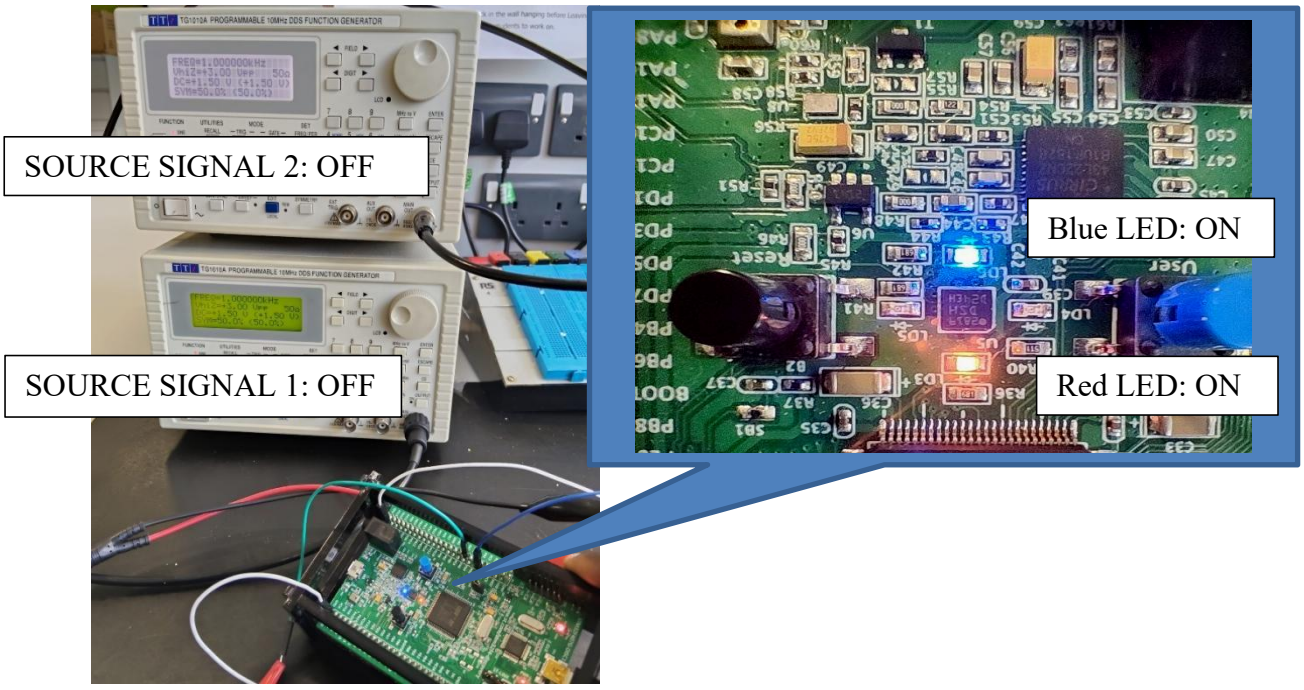


Figure 6 showing the blue and red LED ON indicating half second blink and absence of one or more signal

3.1.5 Data Logging Strategy

Two complementary logging mechanisms satisfy the specification within the microcontroller's memory constraints. As revealed in the same code below, the first captures raw voltage values for over ten signal cycles per channel in float arrays, subsequently exported from Keil uVision for waveform visualisation in Microsoft Excel as shown in Figure 7. The Vpp, mean, variance, standard deviation and RMS are logged every second on the Keil uVision watch window for both channels, the values are recorded and visualized in Microsoft Excel. This logging architecture satisfies the specification while remaining within the 192 KB SRAM budget of the STM32F4.

```
FUNC void save_array() {
    int array_length;
    int idx;
array_length = 200;
    exec("log > STM_Task1_results.csv");
         for (idx = 0; idx < array_length; idx++) {
            printf("%d,%d,%6.4f,%6.4f\n",
                  ADC_DATA1[idx],ADC_DATA2[idx],Volt1[idx],Volt2[idx] );
         }
         exec("log off");
}
```

3.1.6 Results and Validation

All nine input signal combinations were experimentally validated using 1 kHz sinusoidal and square wave inputs at 3 V peak-to-peak with a 1.5 V DC offset. Keil uVision watch-window outputs confirmed correct signal classification, accurate statistical values, and correct LED state transitions across all scenarios. The complete watch-window screenshots for all nine scenarios are provided in Appendix A as summarised in Table A1.

Table A1: Test Scenarios and Watch-window Screenshots

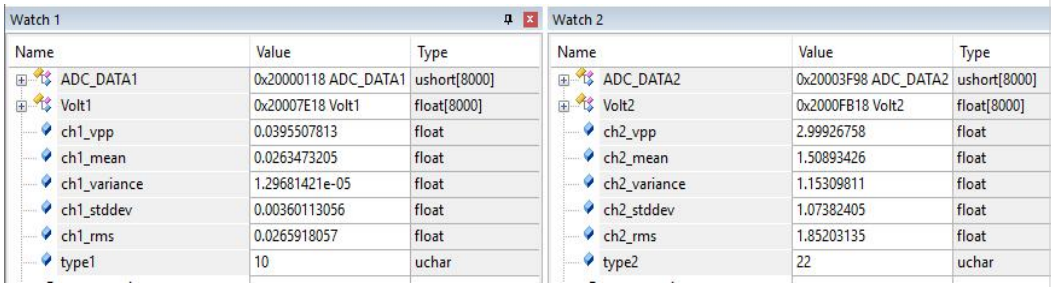
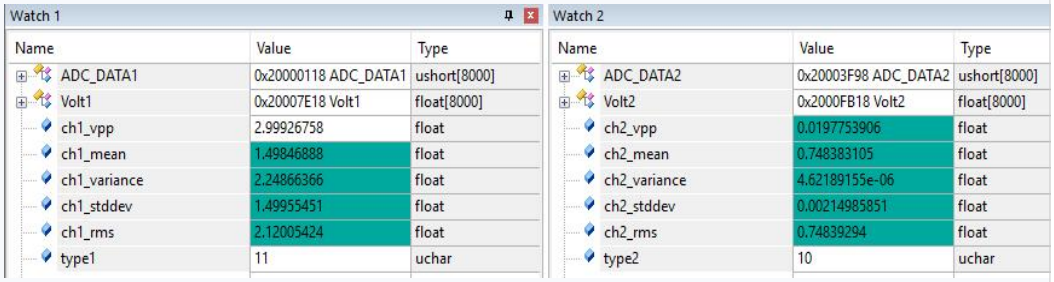
Scenario	Keil Watch-window Screenshot
Sinuosoidal & Square	Watch 1

Watch 1

Name	Value	Type
ADC_DATA1	0x20000118 ADC_DATA1	ushort[8000]
Volt1	0x20007E18 Volt1	float[8000]
ch1_vpp	2.99926758	float
ch1_mean	1.50944126	float
ch1_variance	1.1484344	float
ch1_stddev	1.07165027	float
ch1_rms	1.85118198	float
type1	22	uchar

Watch 2

Name	Value	Type
ADC_DATA2	0x20003F98 ADC_DATA2	ushort[8000]
Volt2	0x2000FB18 Volt2	float[8000]
ch2_vpp	2.99926758	float
ch2_mean	1.49917865	float
ch2_variance	2.24877286	float
ch2_stddev	1.49959087	float
ch2_rms	2.12058163	float
type2	11	uchar

Scenario	Keil Watch-window Screenshot
No signal & Sinusoidal	
Square & No signal	

Representative results confirmed:

- Scenario 3 (Sinusoidal and Square): green LED ON, type1 = 22, type2 = 11;
- Scenario 6 (No signal and Sinusoidal): red LED ON, type1 = 10, type2 = 22;
- Scenario 7 (Square and No Signal): red LED ON, type1 = 11, type2 = 10;

Voltage conversion accuracy was validated by exporting ADC sample values and computed voltages for the first ten cycles, with key reference mappings confirming the correctness of the conversion pipeline. Sample results are shown in Table 2 and plotted in Figure 9.

Table 2: ADC Sample Values & Voltage Conversion Results

ADC_DATA1 (Square)	ADC_DATA2 (Sine)	Volt1 (V)	Volt2 (V)
4095	851	2.9993	0.6233
4095	93	2.9993	0.0681
4095	196	2.9993	0.1436
0	1396	0	1.0225
0	2982	0	2.1841
0	4026	0	2.9487
0	3924	0	2.874
4095	2724	2.9993	1.9951
4095	1126	2.9993	0.8247
4095	851	2.9993	0.6233

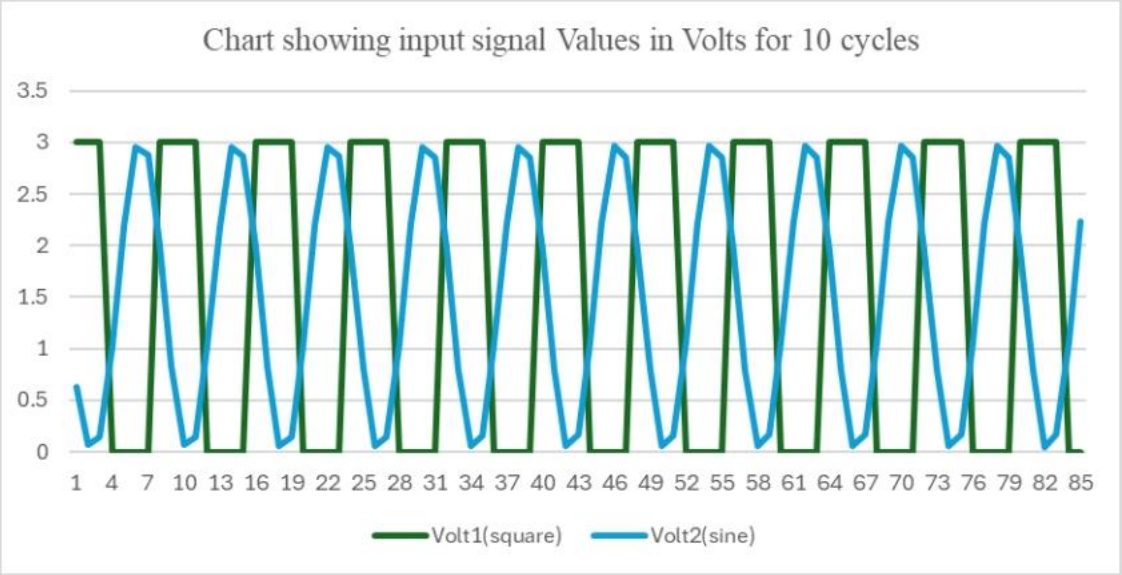


Figure 7: Exported first-ten-cycle voltage values plotted in Microsoft Excel confirming correct ADC scaling for both channels

3.1.7 Critical Evaluation

The system achieves deterministic real-time performance through timer-driven interrupt-driven sampling, predictable memory utilisation through static allocation, and correct waveform classification across all nine validated scenarios. Validation against the Keil watch window and the exported Excel plots confirms both functional correctness and accuracy of the conversion pipeline. However, the design has notable limitations. No embedded security layer, ADC polling within the ISR introduces CPU overhead that scales with sample rate; DMA-based acquisition would eliminate this, reduce interrupt latency, and improve scalability. The threshold-based classifier is sensitive to noise and amplitude drift, and a digital pre-filter would improve noise immunity. Fixed buffer sizes reduce adaptability to different sampling frequencies without firmware modification. These limitations notwithstanding, the system fully meets all specified requirements and demonstrates a well-structured, resource-aware real-time embedded architecture.

3.2 Task 2: Signal Separation Filter Design and Implementation

3.2.1 Problem Formulation and Design Requirements

Task 2 addresses the separation of two sinusoidal components, a 2 kHz primary navigation signal and a 16 kHz auxiliary telemetry signal, combined into a single mixed transmission channel as shown in Figure 8. This reflects a realistic embedded systems scenario common in drone sensor platforms and IoT systems where multiple data streams are multiplexed to reduce transmission bandwidth. The task imposes a dual constraint: the filters must achieve clean, low-distortion signal separation while remaining suitable for battery-powered embedded deployment where computational efficiency, memory footprint, and power consumption are primary design drivers. The three competing priorities are signal fidelity (clean extraction with minimal distortion), computational efficiency (minimum arithmetic operations per sample), and resource constraints (minimal filter order, memory, and power consumption). A sampling frequency of 96 kHz was selected to satisfy the Nyquist criterion with headroom above the 16 kHz component while remaining achievable on the STM32F4.

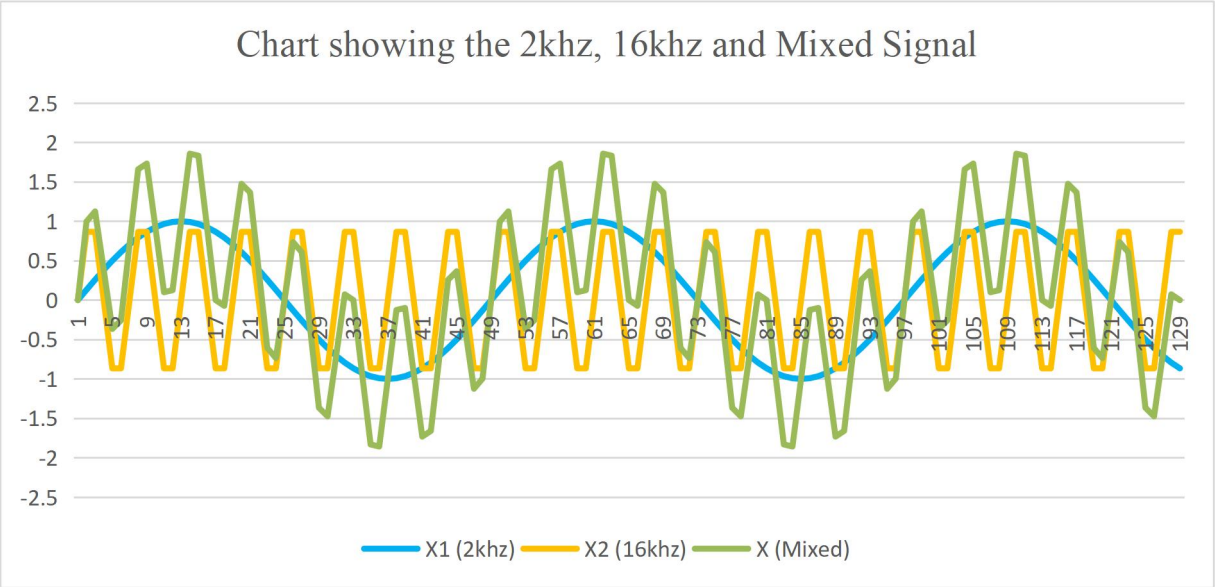


Figure 8 showing the 2 kHz, 16 kHz and Mixed Signal Plot

The filter design process was carried out using MATLAB Filter Designer. The required low-pass and high-pass specifications were created, exported to MATLAB for coefficient generation, and then converted into C-compatible filter structures. These coefficients were imported into Keil uVision, where the filters were implemented and used to generate CSV outputs for visual verification of the separated 2 kHz and 16 kHz signals.

3.2.2 Filter Design Methodology: FIR to IIR

3.2.2.1 Initial FIR Exploration

The design began with FIR filters for their well-known linear phase response, unconditional stability, and predictable behaviour under finite-precision arithmetic. However, as shown in Figure 9A, an unoptimised FIR low-pass filter targeting 80 dB stopband attenuation required order 23, confirmed in Figure 9B. Reducing attenuation to 20 dB achieved order 8 but with

insufficient stopband rejection. Increasing to 30 dB, as shown in Figure 10, raised the order to 9 with unacceptable signal quality.

Figure 9A showing Unoptimised FIR LPF (80 dB)

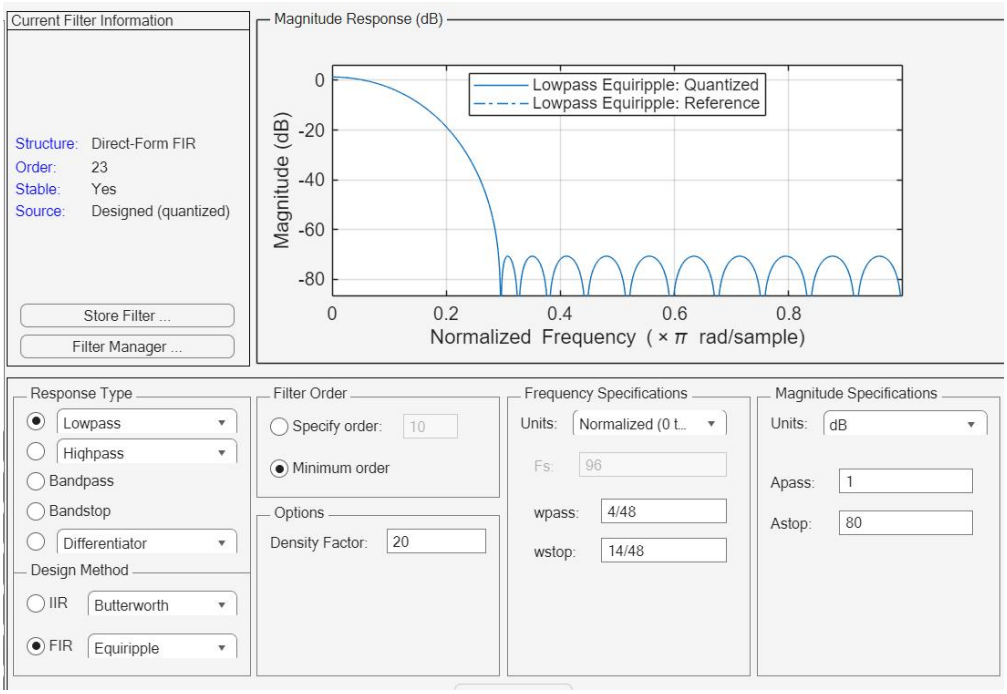


Figure 9B showing the MALAB Filter designer specifications (Order 23)

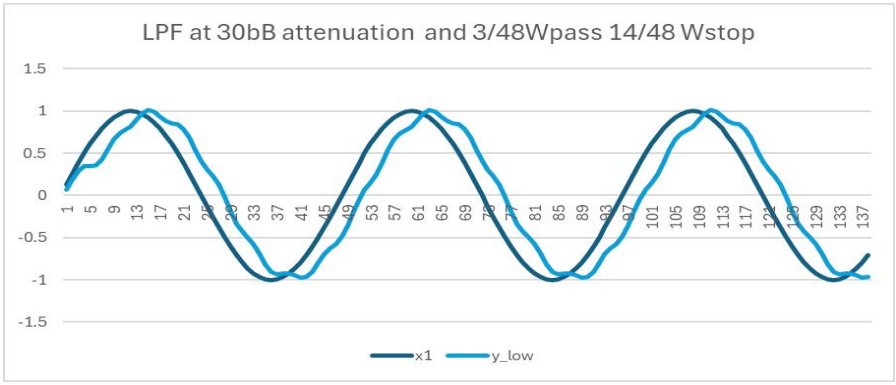


Figure 10 showing FIR LPF at 30 dB Attenuation

For the high-pass filter, Figure 11A and 11B shows that a 20 dB FIR design produced order 8 with observable ripple and edge distortion exceeding the normalised unit level on the response plot, as shown in figure 12 increasing attenuation to 30 dB raised the order to 12, making the design computationally heavy for battery-powered deployment. These results confirmed that FIR designs, while theoretically attractive, do not provide an optimal engineering solution for this task.

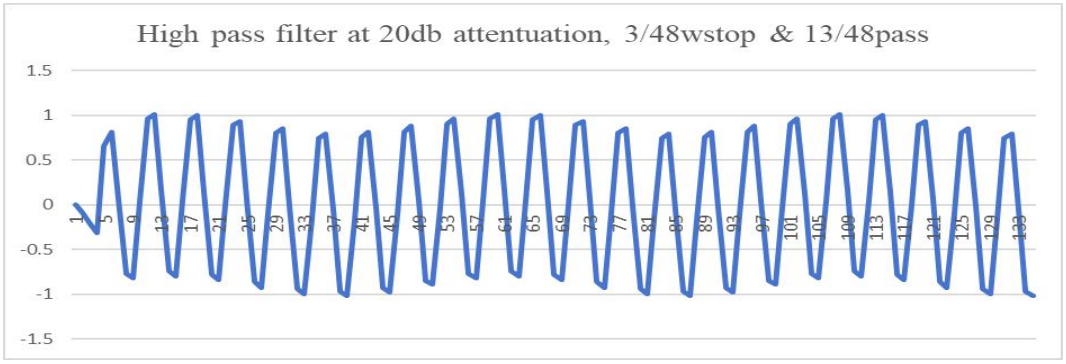


Figure 11 showing FIR HPF at 20 dB Attenuation

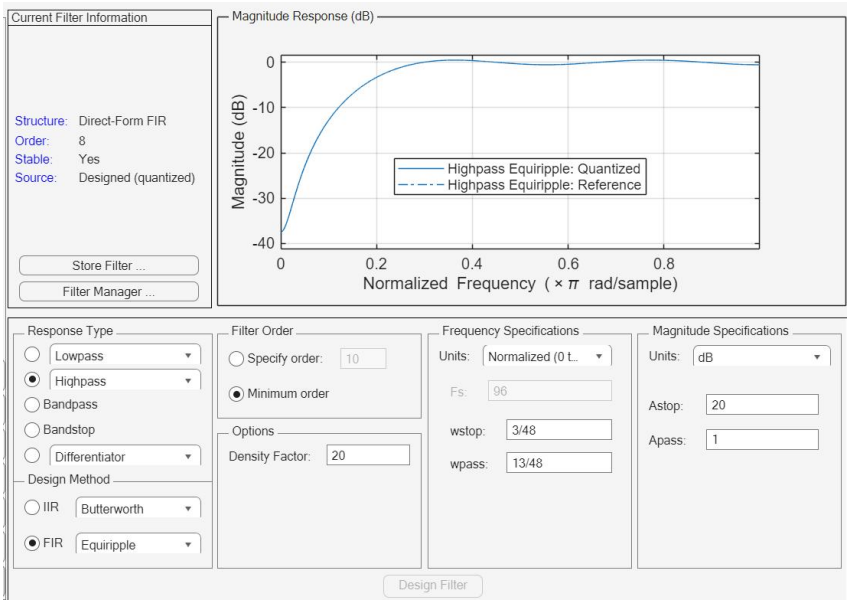


Figure 11B showing FIR HPF Filter Designer Specs (Order 8)

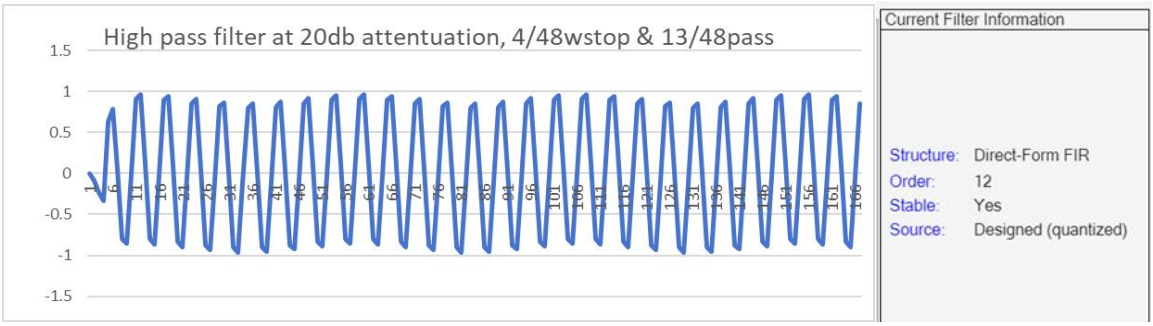


Figure 12 FIR HPF at 30 dB Attenuation (Order 12)

3.2.2.2 IIR Low-Pass Filter for 2 kHz Extraction

Transitioning to IIR design, a Butterworth low-pass filter was iteratively optimised using passband and stopband edges of 3/48 and 15/48 of the sampling frequency. As shown in Figure 13A and 13B, an initial IIR design at 10 dB attenuation achieved order 2, providing a dramatic reduction unlike the FIR design. A 30 dB design shown in Figure 14 achieved order 3, though with higher passband ripple than required. After confirming that 35 dB and 40 dB produce identical filter orders, the attenuation was set to 40 dB as shown in Figure 15A and 15B, yielding a 4th-order IIR filter with a clean, stable passband. The final 4th-order IIR low-pass filter requires only eight multiply-accumulate operations per sample compared to 24 for the order-23 FIR baseline, a three-fold reduction in arithmetic complexity.

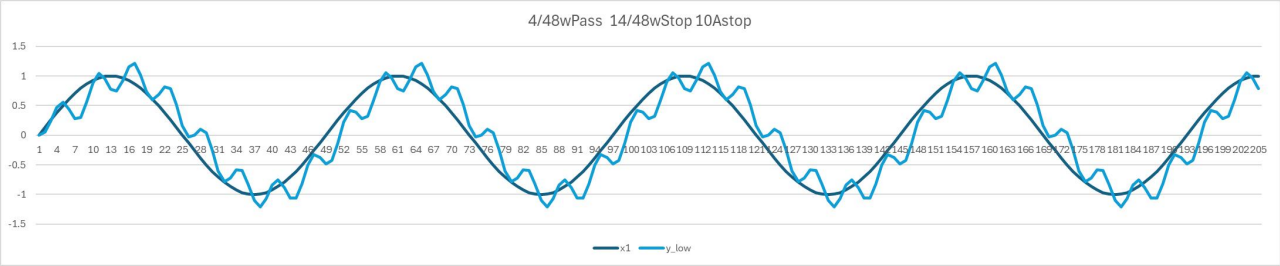


Figure 13A showing IIR LPF at 10 dB Attenuation

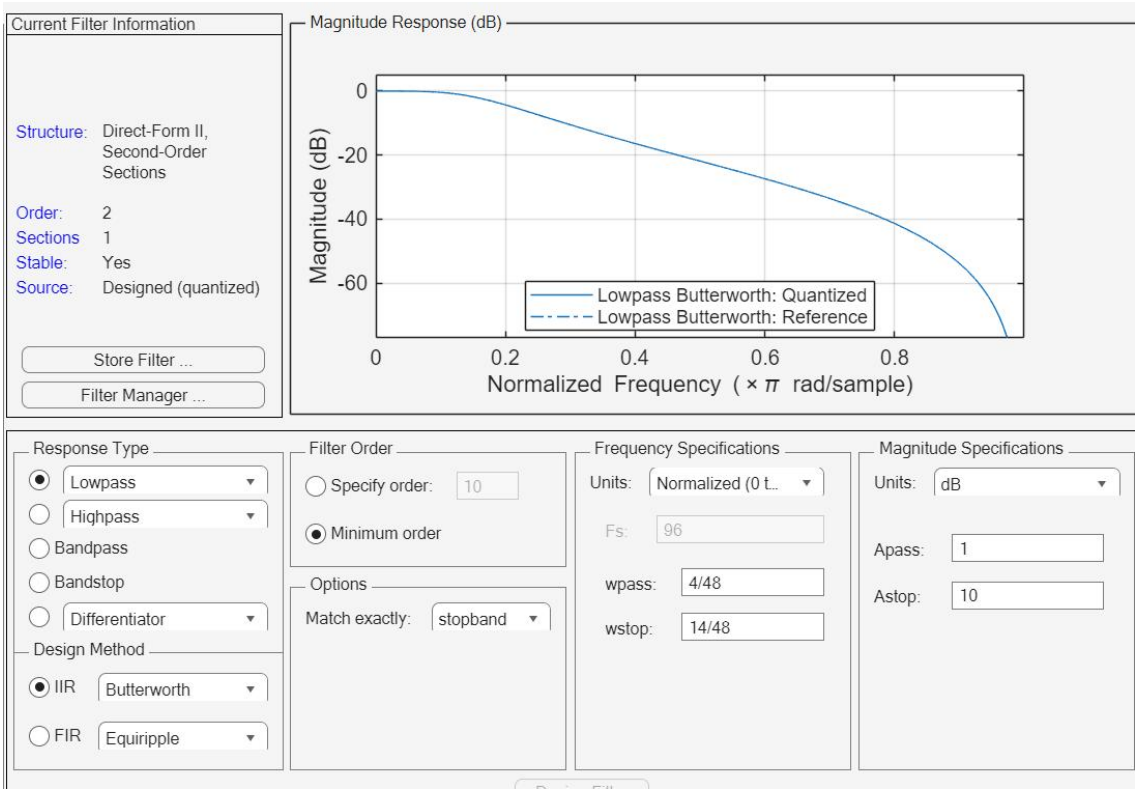


Figure 13B showing the LPF Filter Designer Specs (Order 2)

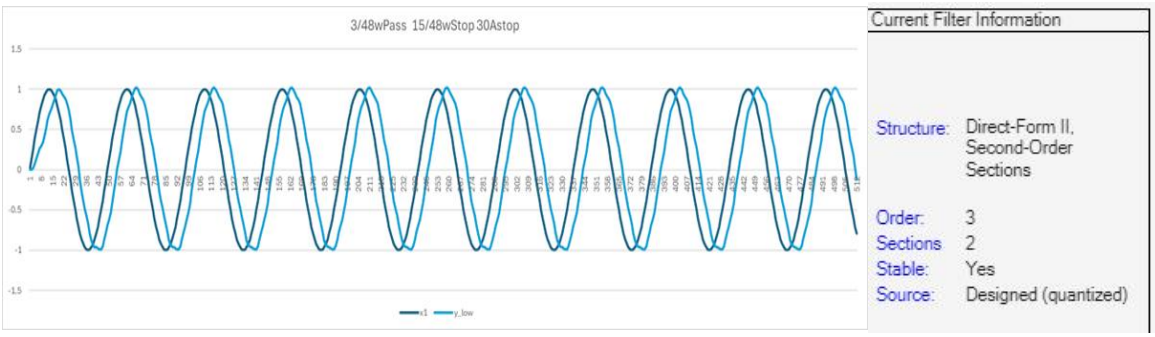


Figure 14 showing the IIR LPF at 30 dB Attenuationand (order 3)

Figure 15A showing IIR LPF at 40 dB Attenuation

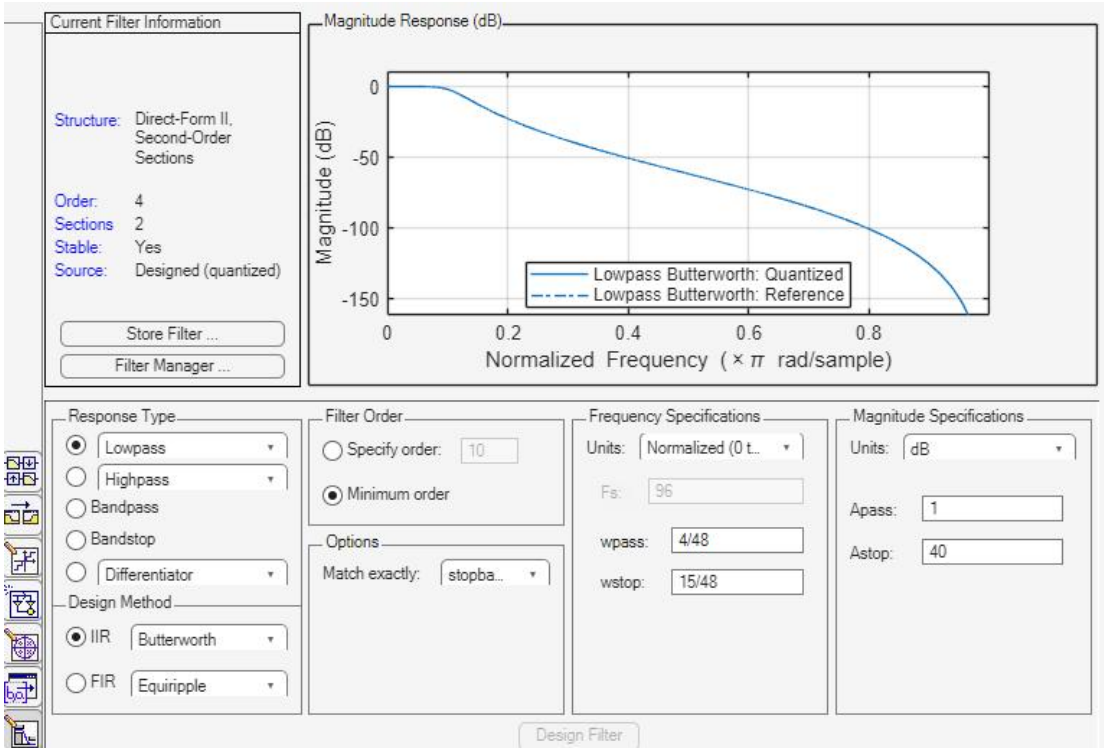


Figure 15B showing IIR LPF Filter Designer Specs (Order 4)

3.2.2.3 IIR High-Pass Filter for 16 kHz Extraction

An analogous iterative process was applied to the high-pass design. As shown in Figure 16A and 16B, an initial IIR high-pass filter at 80 dB attenuation produced order 10, which remained too high. Reducing attenuation to 10 dB as shown in Figure 17A and 17B achieved order 2 producing a clean stable high-pass response that reliably attenuates the 2 kHz component while preserving the 16 kHz signal with full amplitude integrity. The final 2nd-order IIR high-pass filter requires only four multiply-accumulate operations per sample, compared to 13 for the order-12 FIR alternative, a further substantial efficiency gain.

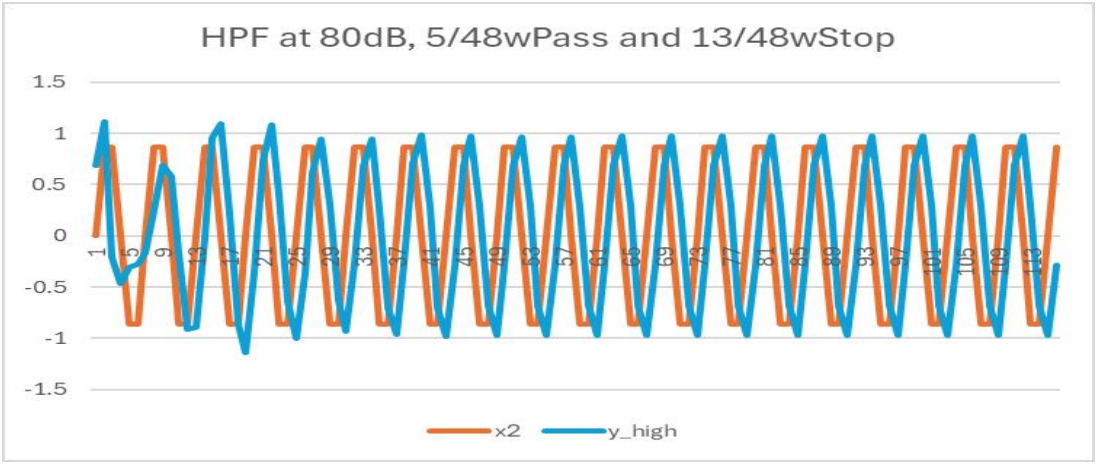


Figure 16A showing IIR HPF at 80 dB Attenuation

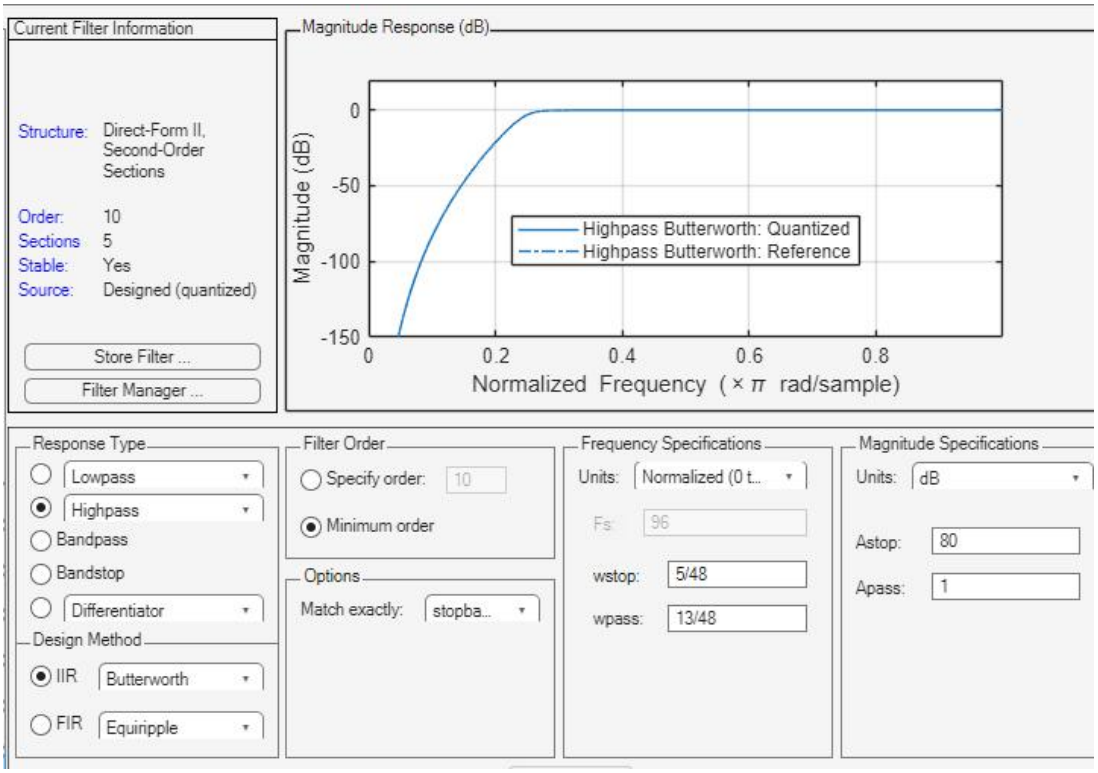


Figure 16B showing the IIR HPF Filter Designer Specs (Order 10)

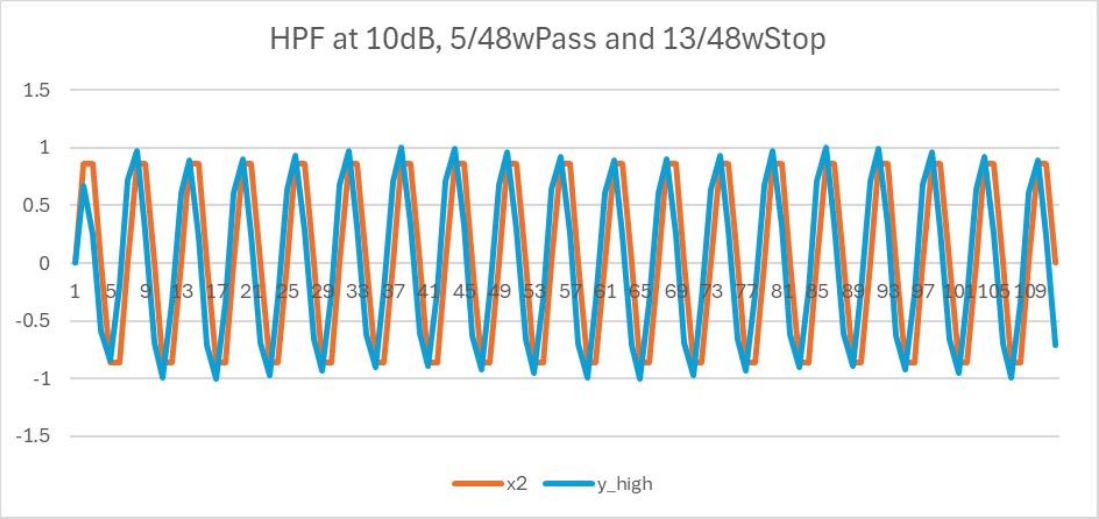


Figure 17A showing IIR HPF at 10 dB Attenuation

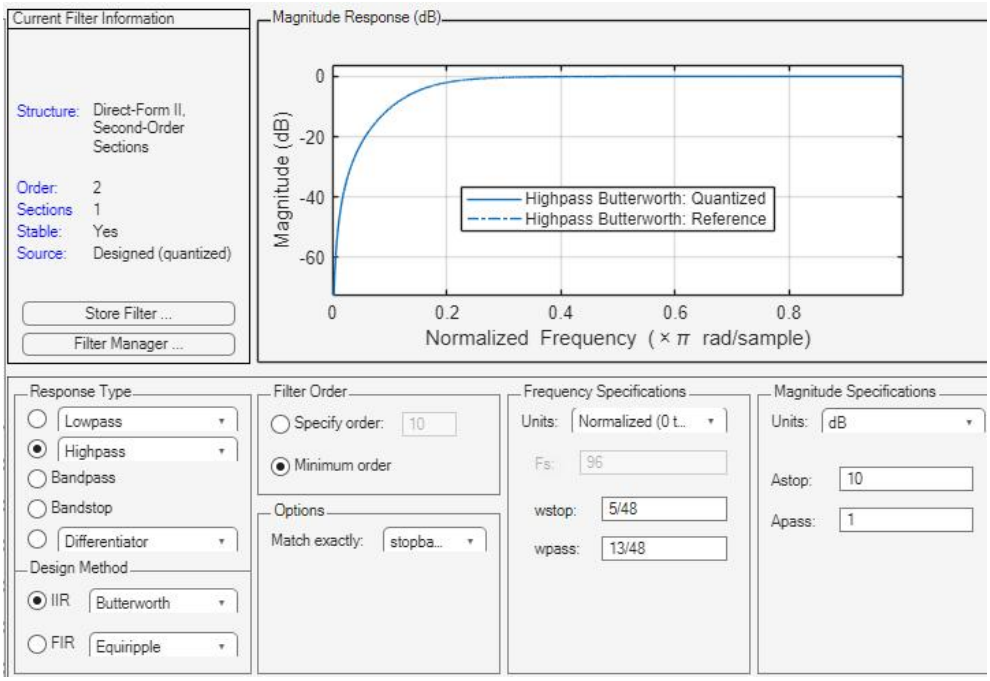


Figure 17B showing the IIR HPF Filter Designer Specs (Order 2)

3.2.3 Results and Validation

The final IIR low-pass filter successfully extracted the 2 kHz navigation signal from the mixed input with clean sinusoidal reconstruction, stable amplitude, and minimal passband ripple, as confirmed by the low-pass filter output shown in Figure 15A and 15B. The final IIR high-pass filter preserved the 16 kHz telemetry waveform with full amplitude integrity while eliminating low-frequency interference, as confirmed by the high-pass filter output shown in Figure 17A and 17B. Both outputs validate the iterative design methodology and confirm that the normalised passband and stopband specifications were correctly formulated for the embedded deployment context.

Table 3: Filter Performance Validation

Filter	Target Signal	Expected Behaviour	Measured Behaviour	Key Metrics	Pass/Fail
FIR LPF (Order 23)	2 kHz	High attenuation	Achieved but computationally heavy	23 MACs/sample	✖ Not suitable
IIR LPF (Order 4)	2 kHz	Extract 2 kHz, suppress 16 kHz	Clean 2 kHz sinusoid recovered	Stopband attenuation ≈ 40 dB; Passband ripple minimal	✔ Pass
FIR HPF (Order 12)	16 kHz	High attenuation	Achieved but inefficient	12 MACs/sample	✖ Not suitable
IIR HPF (Order 2)	16 kHz	Extract 16 kHz, suppress 2 kHz	Full-amplitude 16 kHz preserved	Stopband attenuation ≈ 10 dB; No distortion	✔ Pass
Mixed Signal Separation	2 kHz + 16 kHz	Clean separation	Both signals extracted correctly	No cross-leakage observed	✔ Pass

3.2.7 Critical Evaluation

The iterative IIR filter design methodology produced a solution achieving effective signal separation with minimal computational overhead. Some limitations remain. IIR filters exhibit non-linear phase responses, which can introduce distortion in applications that require phase-coherent reconstruction or synchronised multi-sensor processing. Fixed-point deployment also requires careful management of coefficient quantisation and numerical stability, typically addressed through cascaded biquad structures with appropriate scaling. Although the current implementation uses floating-point arithmetic on the Cortex-M4 FPU, a production system without hardware floating-point support would require fixed-point conversion and full stability validation before deployment.

3.2.4 Energy Efficiency Methodology

Energy efficiency in the filtering subsystem is achieved primarily through the use of low-order IIR filters, which reduce computational workload by nearly 90% compared to the higher-order FIR alternatives originally evaluated. Implementing the filters in Direct-Form II Transposed structure further minimises arithmetic operations and memory usage, consistent with embedded-systems guidance that computational complexity is the dominant driver of dynamic power consumption in microcontroller platforms [3], [6].

The system's timer-driven interrupt architecture enables the processor to sleep between sampling events, avoiding the continuous polling overhead discouraged in low-power cyber-physical system design [3], [5]. Although floating-point operations were used during prototyping, the filter structure is compatible with Q-format fixed-point implementation, which would eliminate FPU usage and reduce each multiply–accumulate to a single-cycle integer operation, this optimisation is widely recommended for energy-constrained automotive and IoT devices [2], [3], [6].

Together, these design choices align with best practices in energy-aware embedded engineering, ensuring that the filtering pipeline remains computationally lightweight while meeting real-time performance requirements.

3.2.5 Societal Benefits of Efficient Signal Processing

Efficient real-time signal processing delivers broad societal benefits across safety-critical, medical, industrial, and communication domains. In autonomous and connected vehicles, lightweight filtering ensures that perception and control systems receive clean sensor data, reducing collision risk and improving the reliability of ADAS and CAV platforms as highlighted in section 2. Similar efficiency requirements appear in IoT and smart-city infrastructures, where low-power filtering enables dense, battery-constrained sensing networks that support mobility management, traffic optimisation, and public-service coordination [2], [5].

Beyond transportation, efficient filtering underpins medical devices such as pacemakers, cochlear implants, and wearable biosignal monitors, where accurate ECG/EEG/EMG extraction must occur under strict energy constraints to extend device lifetime and improve patient outcomes. In industrial automation, frequency-selective filtering enables real-time vibration analysis for predictive maintenance, reducing downtime and enhancing worker safety. Even in drone and robotics applications, reliable separation of navigation and telemetry signals ensures stable flight control and prevents accidents caused by sensor interference. Collectively, these benefits demonstrate how lightweight, real-time signal processing contributes to safer, more reliable, and more sustainable technological ecosystems.

3.2.6 Environmental Risks and Mitigation Strategies

Embedded electronic systems contribute to long-term environmental impact through material consumption, energy use, and eventual e-waste. Hazardous substances in PCBs and components such as lead, cadmium, and brominated flame retardants pose risks when devices are discarded improperly, a concern amplified by the rapidly increasing global e-waste stream. Designing computationally efficient filtering pipelines helps mitigate these effects by lowering power demand, extending device lifetime, and reducing battery waste in distributed IoT and smart-city sensor networks [2], [5], [6].

Sustainability also depends on lifecycle-aware engineering. Regulatory frameworks such as UNECE WP.29 and ISO/SAE 21434 emphasise secure OTA update mechanisms that prolong hardware usability and reduce premature ECU replacement, particularly important in automotive platforms with operational lifetimes of ten to fifteen years [9], [10]. Complementary design strategies such as modular hardware architectures, recyclable PCB materials compliant with RoHS and WEEE requirements, and reduced-hazard manufacturing help minimise end-of-life environmental impact. Energy-harvesting techniques including piezoelectric, thermoelectric and RF harvesting can also reduce reliance on disposable batteries in large IoT deployments. Together, these measures show how environmentally conscious embedded-system design can reduce waste, extend device lifetime and support broader sustainability goals across automotive, industrial and IoT ecosystems.

4. Conclusion and Further Work

4.1 Conclusion

This report has presented the design, implementation, and critical evaluation of two real-time embedded systems alongside a comprehensive literature review of automotive embedded system security, collectively demonstrating how real-time performance requirements, functional safety constraints, cybersecurity considerations, and resource limitations intersect and mutually constrain engineering decisions.

The literature review established that automotive embedded systems face a multi-layered and rapidly evolving security threat landscape spanning in-vehicle network protocol vulnerabilities, connected vehicle remote attack surfaces, ADAS sensor spoofing and adversarial machine-learning threats, and systemic lifecycle challenges. The most mature direction is compositional defence-in-depth security combining lightweight cryptography, hardware roots of trust, AI-based intrusion detection, secure OTA update management, adversarially robust perception, and formal verification, all engineered within real-time, safety-critical, and resource constraints.

Task 1 delivered a fully functional dual-channel signal monitoring and data-logging system achieving deterministic timing, predictable memory utilisation, and accurate waveform classification across all nine validated scenarios, with results confirmed through Keil watch-window outputs, LED hardware observation, and Excel waveform plots. Task 2 demonstrated a structured iterative filter design methodology producing computationally efficient 4th-order IIR low-pass and 2nd-order IIR high-pass filters that achieve effective signal separation with approximately 90 percent reduction in arithmetic operations relative to comparable FIR designs, validated by clean filter output waveforms. The additional discussion of energy efficiency, societal benefits of signal processing, and e-waste environmental risks placed the technical work within its broader engineering and societal context. Together, these tasks reinforce that lightweight, predictable, and resource-aware design strategies are foundational requirements for embedded platforms that must deliver safety, security, and reliability simultaneously under strict constraints.

4.2 Further Work

Several enhancements could improve robustness and scalability. DMA-driven ADC acquisition would replace the current polling approach, eliminating CPU overhead during sample capture, reducing interrupt latency, and enabling higher sampling rates, which is essential for scaling to industrial or automotive-grade monitoring. Adaptive signal classification using FFT-based spectral analysis or machine-learning waveform identification would improve robustness against noise and amplitude drift in real-world environments. For Task 2, conversion to fixed-point arithmetic with cascaded biquad structures would further reduce power consumption and improve numerical stability, with coefficient quantisation effects validated through fixed-point MATLAB simulation. Integration of wireless telemetry via Bluetooth Low Energy or LoRaWAN would transform both systems into deployable remote monitoring solutions.

References

[1] A. A. Mehta et al., "Securing the Future: A Comprehensive Review of Security Challenges and Solutions in Advanced Driver Assistance Systems," IEEE Access, vol. 12, pp. 643-660, 2024.

[2] P. K. Sadhu, V. P. Yanambaka, and A. Abdelgawad, "Internet of Things: Security and Solutions Survey," Sensors, vol. 22, no. 19, art. 7433, 2022.

[3] V. K. Kukkala, S. V. Thiruloga, and S. Pasricha, "Roadmap for Cybersecurity in Autonomous Vehicles," IEEE Consumer Electronics Magazine, vol. 11, no. 6, pp. 13-21, Nov.-Dec. 2022.

[4] R. S. Rathore, C. Hewage, O. Kaiwartya, and J. Lloret, "In-Vehicle Communication Cyber Security: Challenges and Solutions," Sensors, vol. 22, no. 17, art. 6679, 2022.

[5] Z. Wang, H. Wei, J. Wang, X. Zeng, and Y. Chang, "Security Issues and Solutions for Connected and Autonomous Vehicles in a Sustainable City: A Survey," Sustainability, vol. 14, no. 19, art. 12409, 2022.

[6] S. Sonko, E. A. Etukudoh, K. I. Ibekwe, V. I. Ilojianya, and C. D. Daudu, "A Comprehensive Review of Embedded Systems in Autonomous Vehicles: Trends, Challenges, and Future Directions," World Journal of Advanced Research and Reviews, vol. 21, no. 1, pp. 2009-2020, 2024.

[7] M. Krichen, "Formal Methods and Validation Techniques for Ensuring Automotive Systems Security," Information, vol. 14, no. 12, art. 666, 2023.

[8] M. Sadaf et al., "Connected and Automated Vehicles: Infrastructure, Applications, Security, Critical Challenges, and Future Aspects," Technologies, vol. 11, no. 5, art. 117, 2023.

[9] International Organization for Standardization, ISO/SAE 21434:2021 - Road Vehicles: Cybersecurity Engineering. Geneva: ISO, 2021.

[10] United Nations Economic Commission for Europe, UNECE WP.29 Regulation No. 155 - Cyber Security and Cyber Security Management System. Geneva: UNECE, 2021.

Appendix A: Validation Summary — All Nine Signal Scenarios

Table A1 summarises the classification results and LED states validated through Keil uVision watch-window inspection for all nine input combinations. Full screenshots confirming Vpp, Mean, Variance, Standard Deviation, RMS, and signal type codes for each scenario are included below the table.

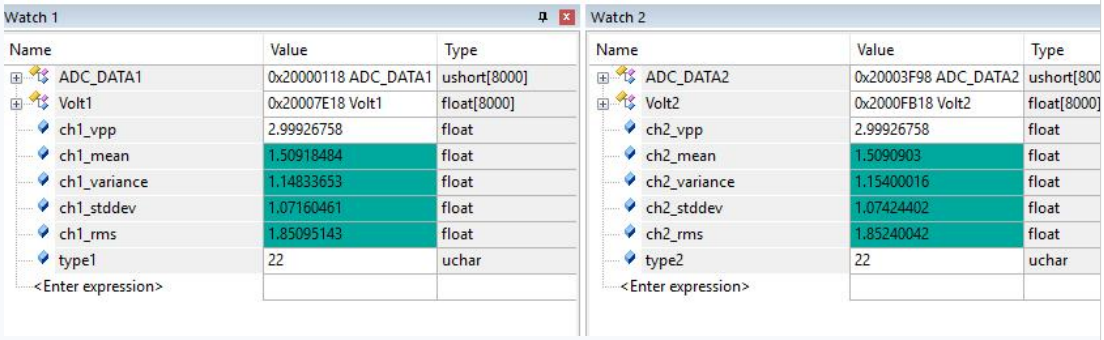
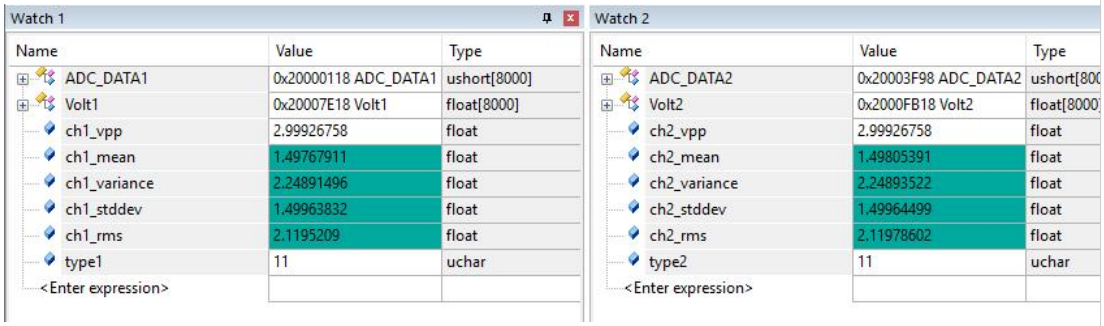
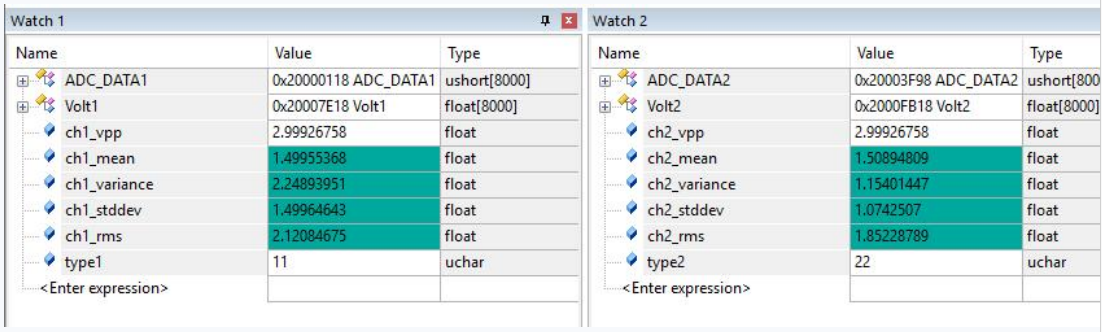
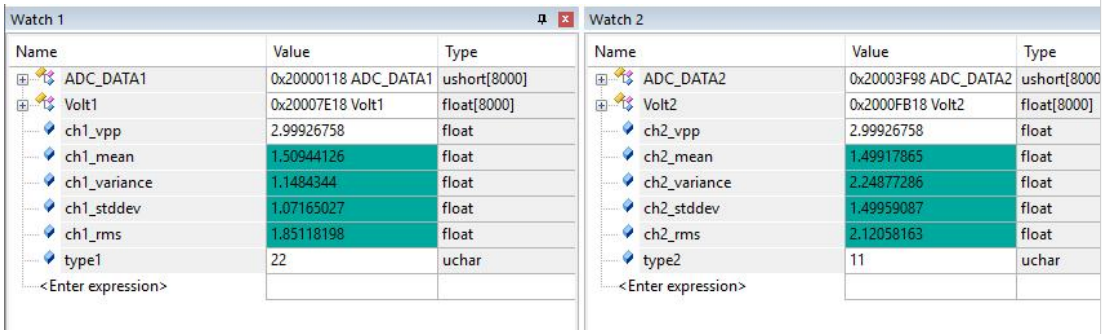
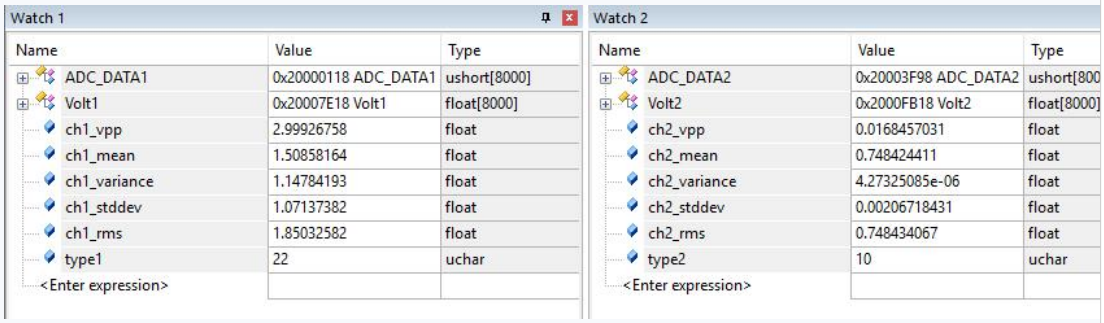
Scenario	Analogue Input 1	Analogue Input 2
1	Sinusoidal signal	Sinusoidal signal
2	Square signal	Square signal
3	Sinusoidal signal	Square signal
4	Square signal	Sinusoidal signal
5	Sinusoidal signal	No signal
6	No signal	Sinusoidal signal
7	Square signal	No signal
8	No signal	Square signal
9	No signal	No signal

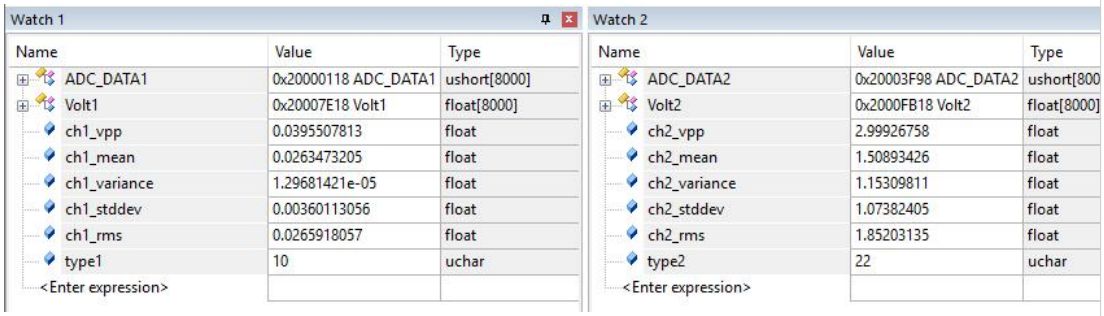
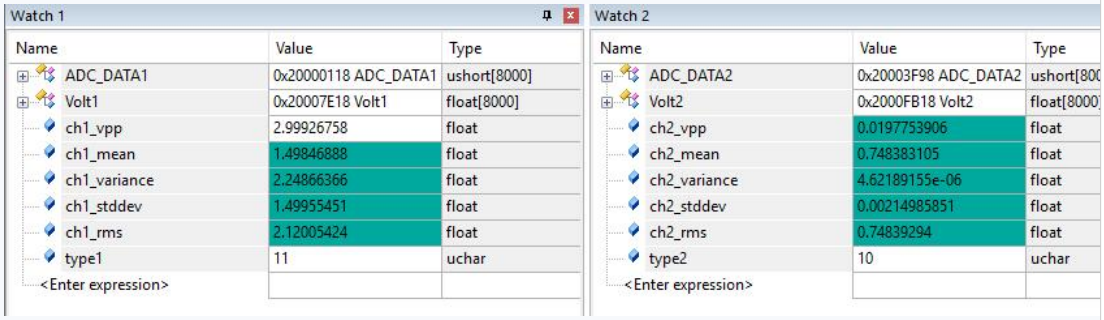
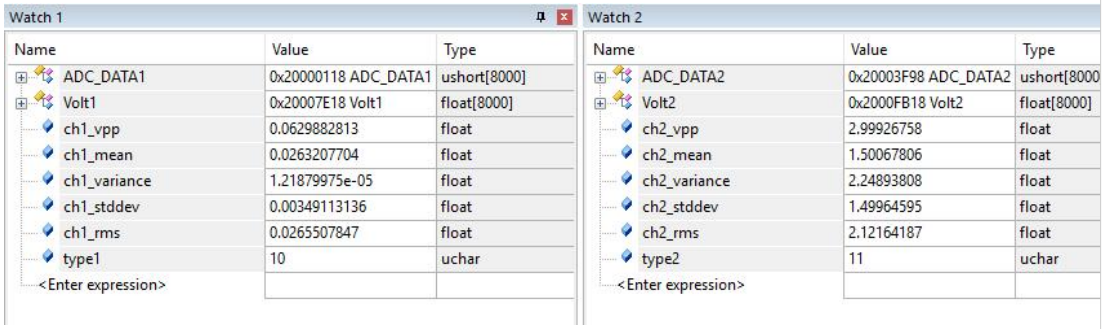
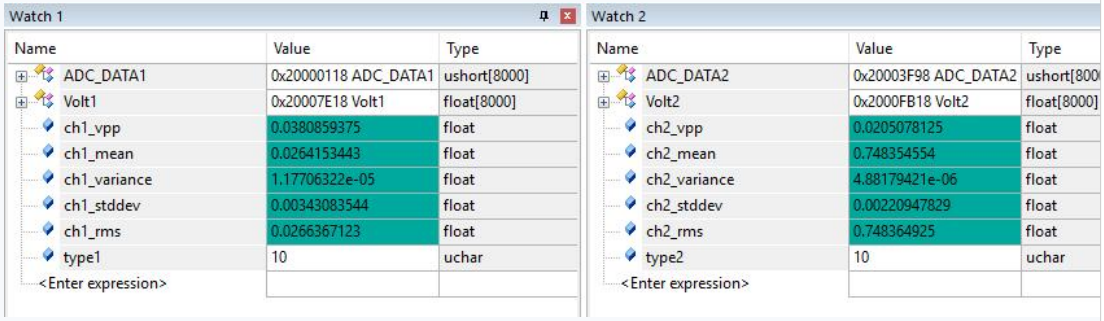
Table 1: Possible combinations of input signals to be considered for system design

Table A2: Keil Watch-Window Validation Summary — All Nine Scenarios

Scenario	Input Ch1	Input Ch2	type1	type2	Green LED	Red LED
1	Sinusoidal	Sinusoidal	22	22	ON	OFF
2	Square	Square	11	11	ON	OFF
3	Sinusoidal	Square	22	11	ON	OFF
4	Square	Sinusoidal	11	22	ON	OFF
5	Sinusoidal	No Signal	22	10	OFF	ON
6	No Signal	Sinusoidal	10	22	OFF	ON
7	Square	No Signal	11	10	OFF	ON
8	No Signal	Square	10	11	OFF	ON
9	No Signal	No Signal	10	10	OFF	ON

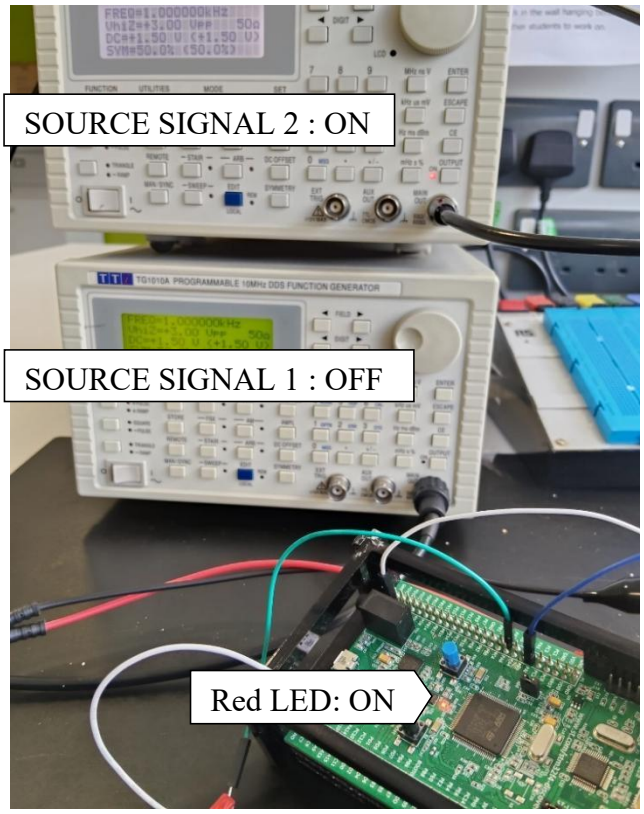
Table A2: Keil Watch-Window Screenshots- All Nine Scenarios

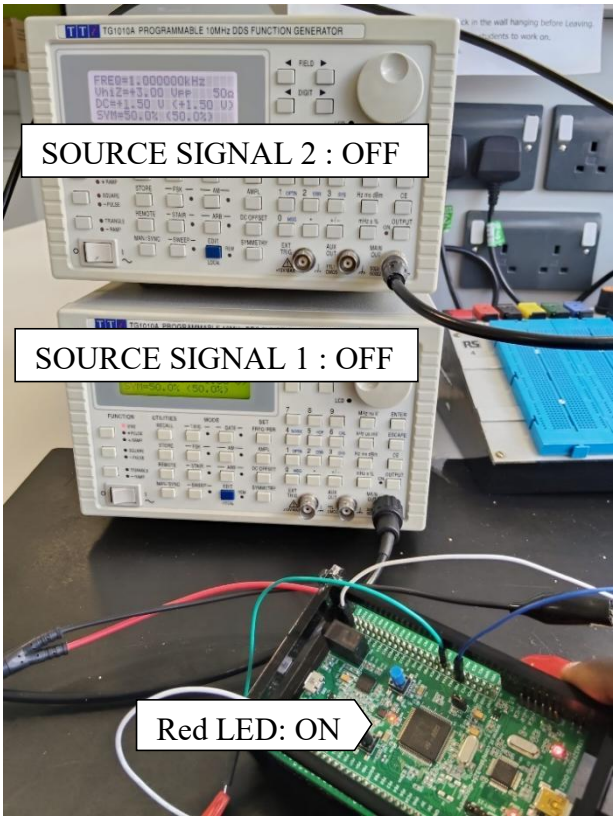
Scenario	Keil Watch-window Screenshot
1. Both sinusoidal signals	
2. Both square signals	
3.Square & sinuosoidal	
4.Sinuosoidal & Square	
5.Sinusoidal & No signal	

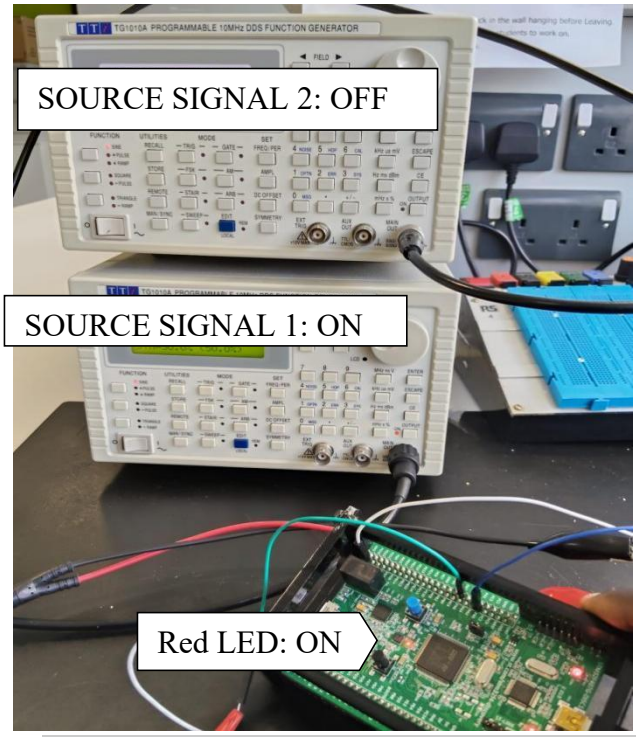
Scenario	Keil Watch-window Screenshot
6. No signal & Sinusoidal signal	
7.Square & No signal	
8.No signal & Square	
9. Both No signal	

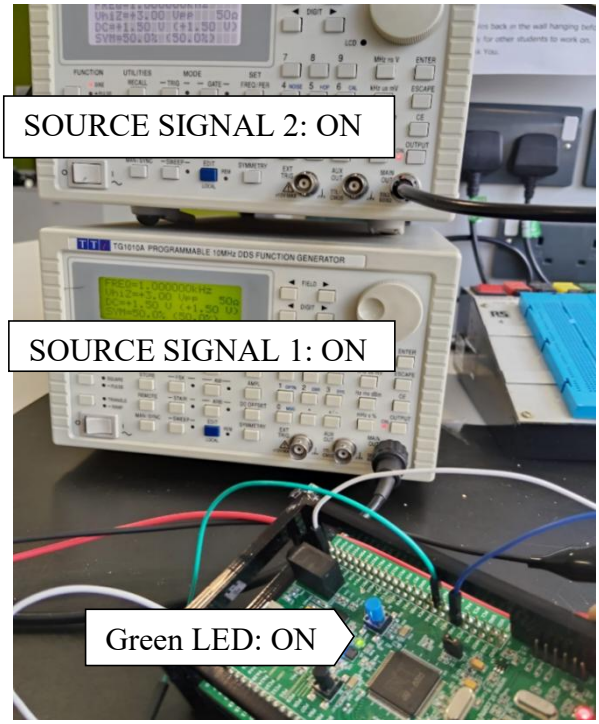
Appendix B: LED Hardware Feedback Images

The images below demonstrate the hardware LED responses for representative signal scenarios observed on the STM32F4 Discovery board during experimental validation. Green LED illumination confirms dual-channel signal presence. Red LED illumination indicates one or both channels absent. The blue LED toggles at 0.5 s intervals in all cases confirming system heartbeat operation. Additional LED response images corresponding to all four distinct LED state conditions are provided as Figures B1 through B4 below.









Appendix C: Full C Source Code — Task 1

The complete C source code is presented below. The code is structured into clearly separated sections for variable declarations, the timer ISR, and signal processing function definitions, with inline commentary throughout.

```
/*  Task 1: Real-Time Signal Monitoring & Data Logging
   STM32F4 Discovery Kit  |  Fs = 8 kHz  |  Two-channel ADC + TIM6  */

#include "main.h"
#include <math.h>

ADC_HandleTypeDef hadc1, hadc2;
TIM_HandleTypeDef htim6;

uint16_t ADC_DATA1[8000], ADC_DATA2[8000];
uint32_t i = 0;
float   Volt1[8000], Volt2[8000];
float   ch1_vpp, ch2_vpp, ch1_mean, ch2_mean;
float   ch1_variance, ch2_variance, ch1_stddev, ch2_stddev;
float   ch1_rms, ch2_rms;
uint8_t  type1, type2;
float   Volt_Log1[10], Volt_Log2[10];
float   Vpp_log1[15], Mean_log1[15], RMS_log1[15];
float   Vpp_log2[15], Mean_log2[15], RMS_log2[15];
uint8_t  log_idx = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
   HAL_ADC_Start(&hadc1);  HAL_ADC_PollForConversion(&hadc1, 1);
   HAL_ADC_Start(&hadc2);  HAL_ADC_PollForConversion(&hadc2, 1);
   if (i < 8000) {
      ADC_DATA1[i] = HAL_ADC_GetValue(&hadc1);
      ADC_DATA2[i] = HAL_ADC_GetValue(&hadc2);
      Volt1[i] = ADC_DATA1[i] * (3.0f / 4096.0f);
      Volt2[i] = ADC_DATA2[i] * (3.0f / 4096.0f);
   }
   HAL_ADC_Stop(&hadc1);  HAL_ADC_Stop(&hadc2);
   i++;
   if (i == 4000) HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_15);  // Heartbeat
   if (i == 8000) {
      i = 0;
      Vpp();  Mean();  Variance_StdDev();  RMS();  Sig_type();
      if (log_idx < 15) {
         Vpp_log1[log_idx]=ch1_vpp;  Mean_log1[log_idx]=ch1_mean;
         RMS_log1[log_idx]=ch1_rms;
         Vpp_log2[log_idx]=ch2_vpp;  Mean_log2[log_idx]=ch2_mean;
         RMS_log2[log_idx]=ch2_rms;  log_idx++;
```

```
        }
    }
    // LED status: Green if both valid, Red if either missing
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12,
        (ch1_vpp>2.5f && ch2_vpp>2.5f) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13,
        (ch1_vpp<2.5f || ch2_vpp<2.5f) ? GPIO_PIN_SET : GPIO_PIN_RESET);
}

void Vpp(void) {
    float max1=Volt1[0],min1=Volt1[0],max2=Volt2[0],min2=Volt2[0];
    for (uint16_t k=1;k<8000;k++) {
        if(Volt1[k]>max1) max1=Volt1[k]; if(Volt1[k]<min1) min1=Volt1[k];
        if(Volt2[k]>max2) max2=Volt2[k]; if(Volt2[k]<min2) min2=Volt2[k];
    }
    ch1_vpp=max1-min1; ch2_vpp=max2-min2;
}

void Mean(void) {
    float s1=0.0f,s2=0.0f;
    for(uint16_t k=0;k<8000;k++){s1+=Volt1[k];s2+=Volt2[k];}
    ch1_mean=s1/8000.0f; ch2_mean=s2/8000.0f;
}

void Variance_StdDev(void) {
    float s1=0.0f,s2=0.0f;
    for(uint16_t k=0;k<8000;k++){s1+=Volt1[k];s2+=Volt2[k];}
    float m1=s1/8000.0f,m2=s2/8000.0f,v1=0.0f,v2=0.0f;
    for(uint16_t k=0;k<8000;k++){
        float d1=Volt1[k]-m1; v1+=d1*d1;
        float d2=Volt2[k]-m2; v2+=d2*d2;
    }
    ch1_variance=v1/8000.0f; ch1_stddev=sqrt(ch1_variance);
    ch2_variance=v2/8000.0f; ch2_stddev=sqrt(ch2_variance);
}

void RMS(void) {
    float s1=0.0f,s2=0.0f;
    for(uint16_t k=0;k<8000;k++){s1+=Volt1[k]*Volt1[k];s2+=Volt2[k]*Volt2[k];}
    ch1_rms=sqrt(s1/8000.0f); ch2_rms=sqrt(s2/8000.0f);
}

void Sig_type(void) {
    float prev1=Volt1[0];
    for(int i=1;i<50;i++){
        float v1=Volt1[i]; float diff1=fabsf(v1-prev1);
        if(ch1_mean<0.9f)            {type1=10; break;}
```

```
        if(diff1>2.9f&&ch1_vpp>2.5f)   {type1=11; break;}
        type1=22; prev1=v1;
    }
    float prev2=Volt2[0];
    for(int i=1;i<50;i++){
        float v2=Volt2[i]; float diff2=fabsf(v2-prev2);
        if(ch2_mean<0.9f)            {type2=10; break;}
        if(diff2>2.9f&&ch2_vpp>2.5f)   {type2=11; break;}
        type2=22; prev2=v2;
    }
}
```